

Mobile Development :

7 : Revision + Introduction to State Management



Professor Imed Bouchrika

National School of Artificial Intelligence
imed.bouchrika@ensia.edu.dz

Outline :

- **Section 1: Revision**

- *Connecting to the network*
- *Singleton/Repo Design Pattern*
- *Relational Mobile Databases*

- **Section 2 : Connecting Widgets**

- *More Design Patterns:*
 - *Publisher-Subscriber Design*
 - *Observer*
- *Case Study : Screen with multiple connected widgets*

- **Section 3 : State Management**

- *GetX*
- *BloC/Cubit*

Section 1

Revision...





Connecting To Web

Revision : Connecting to the Web

- Procedure to get Started

- Install the http package (or you may use the Dio library).

- `flutter pub add http`

- Import the libraries:

- `import 'package:http/http.dart' as http;`

- `import 'dart:convert';`

- For most cases:

- You need to get the data using the “**get**” protocol via : **`http.get(url)`**

Revision : Connecting to the Web

- Getting the data from the Web

```
Future<DATA_TYPE> getNewsData() async {
  print('Function : getNewsFromWeb : Fetching news from web...');
  try {
    final response = await http.get(
      Uri.parse('https://mobilebackendsimpleflask-imed1024-s73qk5q3.leapcell.dev/news.all.get '),
      headers: {'Content-Type': 'application/json'},
    );
    print('Response status: ${response.toString()}');
    if (response.statusCode == 200) {
      final DATA_TYPE data = json.decode(response.body);
      return data ;
    } else {
      throw Exception('Failed to load news: ${response.statusCode}');
    }
  } catch (e, stack) {
    print(' $stack ');
    throw Exception('Error fetching news: $e');
  }
}
```

Revision : Connecting to the Web

- Getting the data from the Web

```
Future<DATA_TYPE> getNewsData() async {
  print('Function : getNewsFromWeb : Fetching news from web...');
  try {
    final response = await http.get(
      Uri.parse('https://mobilebackendsimpleflask-imed1024-s73qk5q3.leapcell.dev/news.all.get '),
      headers: {'Content-Type': 'application/json'},
    );
    print('Response status: ${response.toString()}');
    if (response.statusCode == 200) {
      final DATA_TYPE data = json.decode(response.body);
      return data ;
    } else {
      throw Exception('Failed to load news: ${response.statusCode}');
    }
  } catch (e, stack) {
    print(' $stack ');
    throw Exception('Error fetching news: $e');
  }
}
```

Always **ASYNC**

Use **Error Handling +
Debugging message +
Print the Stack trace**

Revision : Connecting to the Web

- Getting the data from the Web : Encoding

- Returning as a **List<dynamic>**

```
final List<dynamic> data = json.decode(response.body);
```

- Returning as a **List<Map<String,dynamic>>**

```
final List<dynamic> data = json.decode(response.body);  
return data.map((item) => item as Map<String, dynamic>).toList();
```

- Returning as a **Map<String,dynamic>**

```
final Map<String,dynamic> data = json.decode(response.body) as Map<String, dynamic>;
```

Revision : Connecting to the Web



- **Sending the data to the Web**

- You can even send using the **get** method where you embed data within the URL:

- http://address.com/categories.edit?id=6&name=sports&description=some_description

- Recommended way to use the **post** method in order to pass data payload inside the body.

- *Most (may be all) servers impose restrictions on the URL length + Security ?*

Revision : Connecting to the Web

- Sending the data to the Web

```
Future<DATA_TYPE> updateNewsItem(int id, Map<String, dynamic> data) async {
  print('Function: updateNewsItem: Updating news item $id...');
  try {
    final response = await http.post(
      Uri.parse('https://mobilebackendsimpleflask-imed1024-s73qk5q3.leapcell.dev/news/$id/update'),
      headers: {'Content-Type': 'application/json'},
      body: json.encode(data),
    );
    print('Response status: ${response.statusCode}');
    if (response.statusCode == 200) {
      final DATA_TYPE responseData = json.decode(response.body);
      return responseData;
    } else {
      throw Exception('Failed to update news: ${response.statusCode}');
    }
  } catch (e, stack) {
    print('Stack trace: $stack');
    throw Exception('Error updating news: $e');
  }
}
```

Revision : Connecting to the Web

- Sending the data to the Web
 - Binary data ?
 - What's the best practice ?

Revision : Connecting to the Web



- **Issues & Recommendations:**

- Use Postman plugin to test your API endpoints before linking with Flutter
- Handle from the backend side, the Cross-Origin Resource Sharing (CORS) in case your App is to be accessed from a web.
- NEVER HARDCODE the API endpoint addresses inside your code for the UI/Business Logic



Simple Design Patterns

Revision : Singleton Design Pattern

A decorative graphic on the right side of the slide. It features a horizontal pink bar at the top, followed by a dark grey bar. Below these is a long blue bar. Further down, there are several overlapping blue and yellow geometric shapes, including a large blue arrow pointing downwards and to the right, and a yellow rectangle.

- **Singleton Pattern**
 - A technique to ensure only one instance object is created
- **What's a design pattern:**
 - Technique or a solution to a common problem in programming.

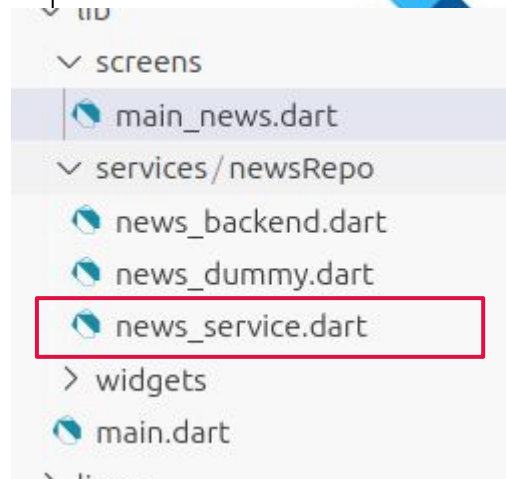
Revision : Singleton Design Pattern

```
import 'package:news_app/services/newsRepo/news_dummy.dart';
```

```
abstract class AbstractNewsRepo {  
  Future<List<Map>> getNewsData();  
}
```

```
static AbstractNewsRepo? _newsInstance;  
  
static AbstractNewsRepo getInstance() {  
  _newsInstance ??= NewsDummy();  
  return _newsInstance!; // For backend data  
}
```

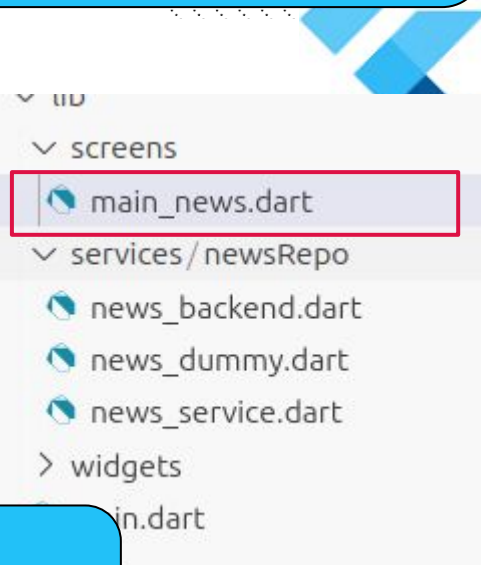
```
}
```



Revision : Singleton Design Pattern

We are decoupling this class from the Backend or dummy dart files,
(We are not importing them)

```
class _NewsScreenState extends State<NewsScreen> {  
  late Future<List<Map>> news;  
  late AbstractNewsRepo newsRepo;  
  @override  
  void initState() {  
    super.initState();  
    newsRepo = AbstractNewsRepo.getInstance();  
    news = newsRepo.getNewsData();  
  }  
}
```



Can we just simply say :

```
newsRepo = new BackendNews(); ?
```

In other files ? we may end up recreating more objects ?..

Revision : Factory Design Pattern



- **Factory Pattern**

- Same as the singleton, but does not ensure only one active instance is running.
- It is a creational design pattern providing an interface for creating objects with the possibility to :
 - Choose the concrete type from the abstraction.
 - Add further business logic when creating an object.
 - Remove totally the “new” and reduce the level of coupling (= **decoupling**)

Revision : Factory Design Pattern

```
import 'package:news_app/services/newsRepo/news_dummy.dart';
```

```
abstract class AbstractNewsRepo {  
  Future<List<Map>> getNewsData();  
}
```

```
static AbstractNewsRepo getInstance() {  
  AbstractNewsRepo _newsInstance = NewsDummy();  
  _newsInstance.setNewsSize(10);  
  _newsInstance.setDefaultLanguage('en');  
  return _newsInstance!; // For backend data  
}
```

Design Pattern : Provider

- **Provider**

- The Provider Design Pattern is a structural design pattern that facilitates the **provision** of objects, services or dependencies to **other classes** or software components in a controlled, flexible, and reusable manner.

Design Pattern : Service Locator



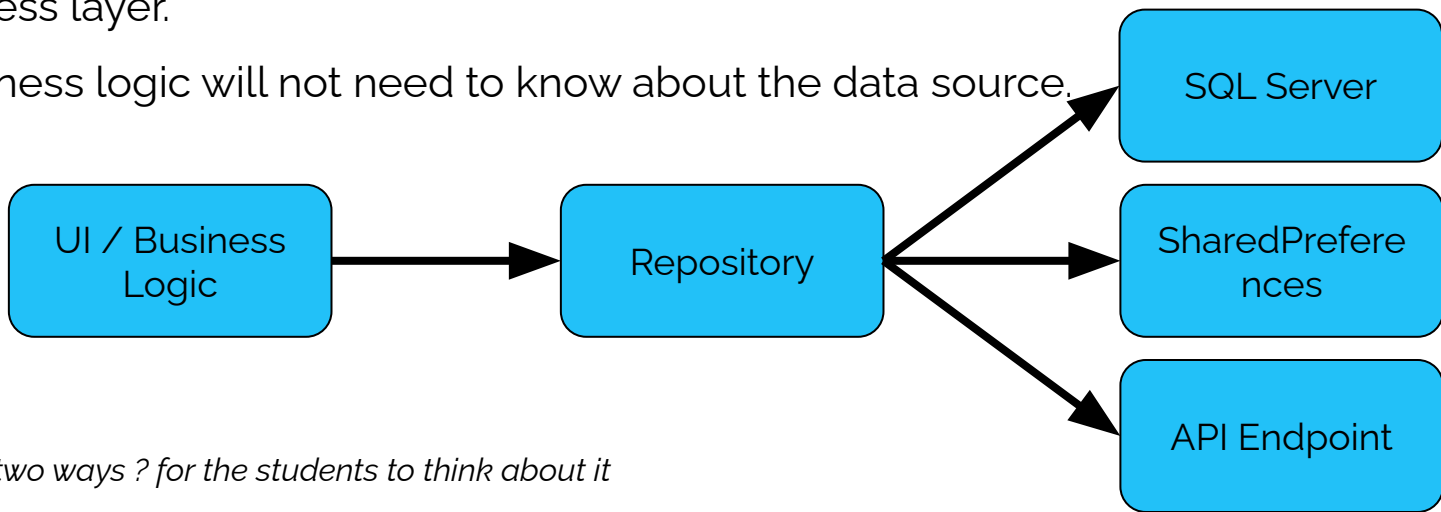
- **Service Locator Pattern**

- is a design pattern used to encapsulate the process of obtaining services. It provides a centralized registry or locator where consumers (clients) can query and retrieve the services they need, often by type or name.

Revision : Repository Design Pattern

- **Repository Pattern**

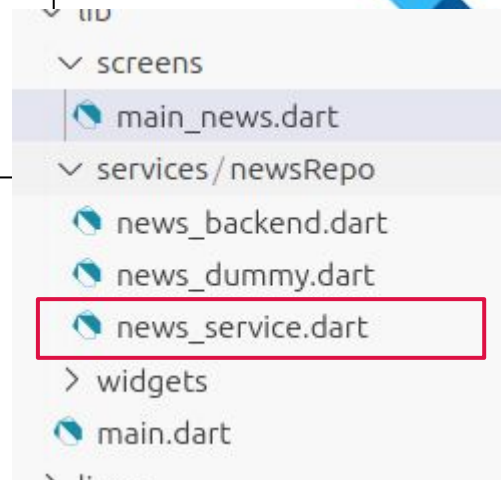
- A technique that acts as an abstraction layer between the business logic and the data access layer.
- The business logic will not need to know about the data source.



One way communication or two ways ? for the students to think about it

Revision : Repository Design Pattern

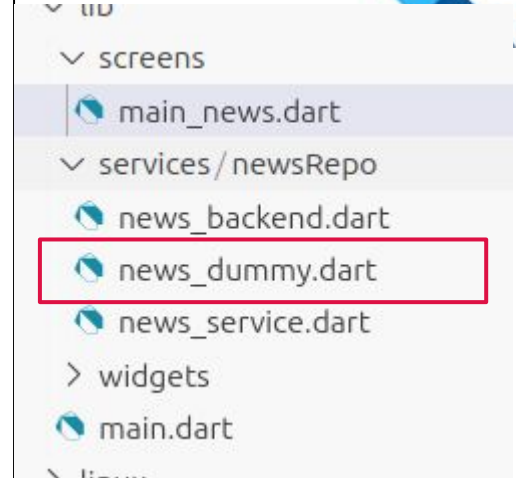
```
abstract class AbstractNewsRepo {  
    Future<List<Map>> getNewsData();  
}
```



```

class NewsDummy extends AbstractNewsRepo {
  @override
  Future<List<Map>> getNewsData() async {
    await Future.delayed(Duration(seconds: 5)); // Simulate network delay
    List<Map> news = [
      {
        "id": 1,
        "title":
          "Universities Adopt AI-Powered Learning Platforms for Personalized
Education",
        "image_url":
          "https://images.unsplash.com/photo-1501504905252-473c47e087f8" ,
        "description":
          "Major universities across the country are implementing artificial
intelligence systems to create personalized learning experiences for students,
adapting course content based on individual progress and learning styles." ,
        "category_id": 3,
      },
      {
        "id": 2,
        "title":
          "Study Reveals Benefits of Bilingual Education in Early Childhood" ,
        "image url":
          "https://images.unsplash.com/photo-1503676260728-1c00da094a0b" ,
        "description":
          "New research shows that children enrolled in bilingual education
programs demonstrate enhanced cognitive flexibility and problem-solving skills
compared to monolingual peers." ,
        "category_id": 3,
      },
    ];
    return news;
  }
}

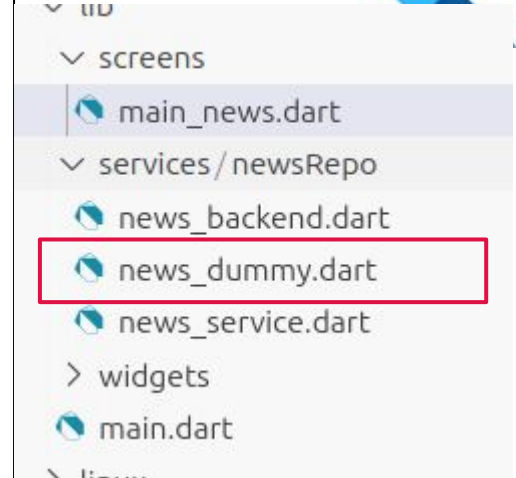
```



```

class NewsDummy extends AbstractNewsRepo {
  @override
  Future<List<Map>> getNewsData() async {
    await Future.delayed(Duration(seconds: 5)); // Simulate network delay
    List<Map> news = [
      {
        "id": 1,
        "title":
          "Universities Adopt AI-Powered Learning Platforms for Personalized
Education",
        "image_url":
          "https://images.unsplash.com/photo-1501504905252-473c47e087f8" ,
        "description":
          "Major universities across the country are implementing artificial
intelligence systems to create personalized learning experiences for students,
adapting course content based on individual progress and learning styles." ,
        "category_id": 3,
      },
      {
        "id": 2,
        "title":
          "Study Reveals Benefits of Bilingual Education in Early Childhood" ,
        "image url":
          "https://images.unsplash.com/photo-1503676260728-1c00da094a0b" ,
        "description":
          "New research shows that children enrolled in bilingual education
programs demonstrate enhanced cognitive flexibility and problem-solving skills
compared to monolingual peers." ,
        "category_id": 3,
      },
    ];
    return news;
  }
}

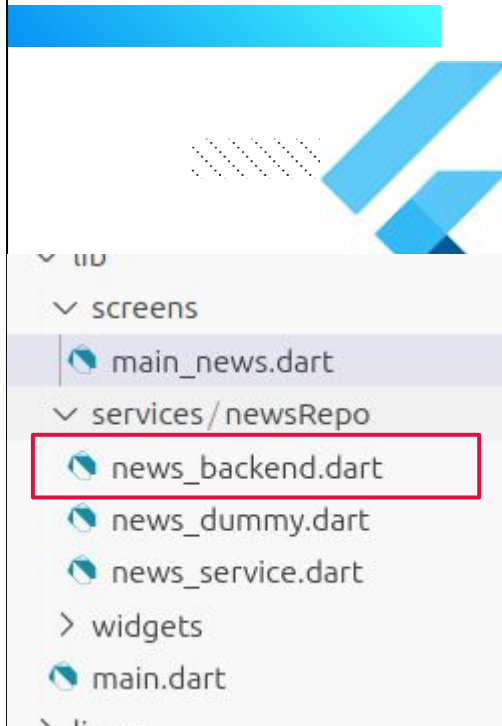
```



flutter pub add http

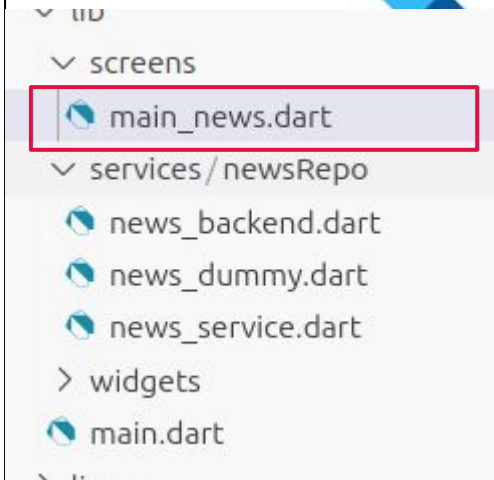
```
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:news_app/services/newsRepo/news_service.dart';

class NewsBackend extends AbstractNewsRepo {
  @override
  Future<List<Map>> getNewsData() async {
    print('Function : getNewsFromWeb : Fetching news from web...');
    try {
      final response = await http.get(
        Uri.parse(
          'https://mobilebackendsimpleflask-imed1024-s73qk5q3.leapcell.dev/news.al
1.get ',
        ),
        headers: {'Content-Type': 'application/json'},
      );
      print('Response status: ${response.toString()}');
      if (response.statusCode == 200) {
        // Parse JSON response
        final List<dynamic> data = json.decode(response.body);
        return data.map((item) => item as Map<String, dynamic>).toList();
      } else {
        throw Exception('Failed to load news: ${response.statusCode}');
      }
    } catch (e) {
      throw Exception('Error fetching news: $e');
    }
  }
}
```



Revision : Repository Design Pattern

```
class _NewsScreenState extends State<NewsScreen> {  
  late Future<List<Map>>> news;  
  late AbstractNewsRepo newsRepo;  
  @override  
  void initState() {  
    super.initState();  
    newsRepo = AbstractNewsRepo.getInstance();  
    news = newsRepo.getNewsData();  
  }  
}
```

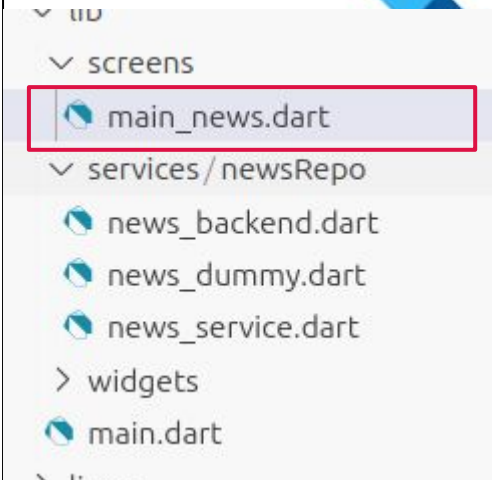


Revision : Repository Design Pattern

```
class _NewsScreenState extends State<NewsScreen> {  
  late Future<List<Map>> news;  
  late AbstractNewsRepo newsRepo;  
  @override  
  void initState() {  
    super.initState();  
    newsRepo = AbstractNewsRepo.getInstance();  
    news = newsRepo.getNewsData();  
  }  
}
```

Is the news loaded from the Web ? Dummy data ? from a local DB ? from a Cache ?

This is not the concern of the this screen





Storage in Mobile Apps

```

import 'package:flutter/material.dart' ;
class Increment extends StatefulWidget {
  const Increment ({super.key});
  @override
  State<Increment> createState () => _IncrementState ();
}
class _IncrementState extends State<Increment> {
  var increment_value = 0;
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Container(
        child: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Text ('$increment_value'),
              ElevatedButton(
                onPressed: () {
                  increment_value++;
                  setState(() {});
                },
                child: Text('Increment'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
bool incrementValue (
  increment value++;
  setState(() {});
  return true;
)

```

Import libraries + define shared_ref Instance

01

```

import 'package:flutter/material.dart' ;
import 'package:shared_preferences/shared_preferences.dart';

class Increment extends StatefulWidget {
  const Increment ({super.key});

  @override
  State<Increment> createState () => _IncrementState ();
}

class _IncrementState extends State<Increment> {
  var increment_value = 0;

  late final SharedPreferences shared_prefs;

```

```

import 'package:flutter/material.dart';
class Increment extends StatefulWidget {
  const Increment({super.key});
  @override
  State<Increment> createState () => _IncrementState ();
}
class _IncrementState extends State<Increment> {
  var increment_value = 0;
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Container(
        child: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Text('$increment_value'),
              ElevatedButton(
                onPressed: () {
                  increment_value++;
                  setState(() {});
                },
                child: Text('Increment'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

bool incrementValue() {
  increment_value++;
  setState(() {});
  return true;
}

```

Initialize + load data

02

```

class _IncrementState extends State<Increment> {
  var increment_value = 0;

  late final SharedPreferences shared_prefs;

  @override
  void initState() {
    super.initState();
    initCounter();
  }

  Future<bool> initCounter() async {
    shared_prefs = await SharedPreferences.getInstance();
    increment_value = shared_prefs.getInt('increment_value') ?? 0;
    return true;
  }
}

```

```

import 'package:flutter/material.dart';
class Increment extends StatefulWidget {
  const Increment({super.key});
  @override
  State<Increment> createState () => _IncrementState ();
}
class _IncrementState extends State<Increment> {
  var increment_value = 0;
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Container(
        child: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Text('$increment_value'),
              ElevatedButton(
                onPressed: incrementValue,
                child: const Text('Increment'),
              ),
            ],
          ),
        ),
      ),
    );
  }
  bool incrementValue () {
    increment_value++;
    setState(() {});
    return true;
  }
}

```

Initialize + load data

03

```

bool incrementValue () {
  increment_value++;
  shared_prefs.setInt('increment_value', increment_value);
  setState(() {});
  return true;
}

```

Data Persistence using Flutter

- For the case that **SharedPreferences** is used in many classes, You can initialize it within the main file (NOT RECOMMENDED)

```
late final SharedPreferences shared_prefs;

Future<bool> init_my_app() async {
  shared_prefs = await SharedPreferences.getInstance();
  return true;
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await init_my_app();
  runApp(const MainApp());
}
```

Data Persis Flu

Instead, we can use
the Service Locator
provided by the
Provider library
OR
OTHER Libraries

```
import 'package:flutter/material.dart';
import 'package:increment shared preference/screens/increment.dart';
import 'package:provider/provider.dart';
import 'package:shared_preferences/shared_preferences.dart';

late final SharedPreferences shared_prefs;

Future<bool> init_my_app() async {
  shared_prefs = await SharedPreferences.getInstance();
  return true;
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await init_my_app();
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [Provider<SharedPreferences>.value(value: shared_prefs)],
      child: MaterialApp(home: Increment()),
    );
  }
}
```

Data Persistence using Flutter

Instead, y
the Servi
provided
Provider
OR
OTHER Libraries

```
Future<bool> initCounter() async {  
    final shared_prefs = Provider.of<SharedPreferences>(context, listen: false);  
    increment_value = shared_prefs.getInt('increment_value') ?? 0;  
    setState(() {});  
    return true;  
}
```

Revision : Relational Databases in Flutter

01

- **Steps to started : Plugins & Packages**

- SQLite is the flutter plugin for using SQLite

- <https://pub.dev/packages/sqlite>

- To get started install:

- sqlite : package provides classes and f

- path : package provides functions to de

- **flutter pub add sqlite path**

- Import the packages:

sqlite 2.3.0

Published 4 months ago • [tekartik.com](https://pub.dev/packages/sqlite) Dart 3 compatible

SDK

FLUTTER

PLATFORM

ANDROID

IOS

MACOS

```
import 'dart:async';

import 'package:flutter/widgets.dart';
import 'package:path/path.dart';
import 'package:sqlite/sqlite.dart';
```

Revision : Relational Databases in Flutter

01

- Steps to started : Plugins & Packages
 - Running App on **Windows and Linux** :

1. Add the following library :

flutter pub add sqflite_common_ffi

2. Initialize the library :

```
import 'package:flutter/material.dart';
import 'package:to_do_app_abstract/screens/todo.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';
import 'dart:io' show Platform;

void main() {
  if (Platform.isLinux || Platform.isWindows) {
    sqfliteFfiInit();
    databaseFactory = databaseFactoryFfi;
  }
  runApp(const MainApp());
}
```

```
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class DBHelper {
  static const database name = "ENSIA_MY_DB.db";
  static const database_version = 4;
  static var database;

  static Future getDatabase() async {
    if (database != null) {
      return database;
    }
    database = openDatabase(
      join(await getDatabasesPath(), _database_name),
      onCreate: (database, version) {
        database.execute('''
          CREATE TABLE todo (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT,
            done INTEGER,
            duedate TEXT,
            create_date TEXT)
        ''');
      },
      version: database version,
      onUpgrade: (db, oldVersion, newVersion) { },
    );
    return database;
  }
}
```

02

```
import 'package:sqflite/sqflite.dart' ;
import 'package:to do app abstract/repo/todo_abstract.dart' ;
import '../databases/dbhelper.dart' ;

class TodoDB extends AbstractTodoRepo {
  @override
  Future<List<Map>> getData() async {
    final database = await DBHelper.getDatabase();
    return database.rawQuery(''SELECT
      todo.id ,
      todo.title,
      todo.done
    FROM todo
    '');
  }
  @override
  Future<bool> deleteItem(int index) async {
    final database = await DBHelper.getDatabase();
    database.rawQuery('"'delete from todo where id=?'"', [index]);
    return true;
  }
  @override
  Future<bool> insertItem(Map<String, dynamic> item) async {
    final database = await DBHelper.getDatabase();
    database.insert("todo", item, conflictAlgorithm: ConflictAlgorithm.replace);
    return true;
  }
  @override
  Future<bool> updateItem(int index, Map<String, dynamic> item) async {
    final database = await DBHelper.getDatabase();
    database.update(index, item);
    return true;
  }
}
```

03

Data Persistence using Flutter

```
onUpgrade: (db, oldVersion, newVersion) async {  
  print('Upgrading database from version $oldVersion to $newVersion');  
  
  if (oldVersion < 2) {  
    await db.execute('ALTER TABLE todo ADD COLUMN priority INTEGER DEFAULT 0');  
    print('Added priority column');  
  }  
  
  if (oldVersion < 3) {  
    await db.execute('ALTER TABLE todo ADD COLUMN category TEXT');  
    print('Added category column');  
  }  
}
```

Section 2

Connecting Widgets





More Design Patterns

Design Pattern : Subscribe-Publish

- **Subscribe-Publish :**

- It is a messaging design pattern where senders (publishers) and receivers (subscribers) communicate indirectly through a centralized broker.
- Publishers emit messages about events or changes without needing to know who will receive them
- ⇒ while subscribers register interest in specific events and are notified by the broker when those events occur.

Design Pattern : Subscribe-Publish

Senders are totally decoupled from the receivers.

They are also decoupled from a temporal level (asynchronous communication)

- **Subscribe-Publish :**

- It is a messaging design pattern where senders (publishers) and receivers (subscribers) communicate indirectly through a centralized broker.
- Publishers emit messages about events or changes without needing to know who will receive them
- ⇒ while subscribers register interest in specific events and are notified by the broker when those events occur.

Design Pattern : Subscribe-Publish

- **Subscribe-Publish :**

- It is a messaging design pattern where Publishers (publishers) and Subscribers (subscribers) communicate indirectly through a central Broker (broker).
- Publishers emit messages about events or changes without needing to know who will receive them
- ⇒ while subscribers register interest in specific events and are notified by the broker when those events occur.

Students who are interested to receive news about AI for Health prepared by me, subscribe to the list called "ai-health" .. when i write a news related to this topic, i publish it to this list, it gets broadcast to all students..



Design Pattern : Observer

- **Observer**

- is a behavioral pattern where an object (**subject**), maintains a list of its observers (dependents)
- ⇒ **notifies** them of any state changes, typically by calling their update methods.
- The subject **must know** their dependents.



States & Widgets

State Management in Flutter

- **Stateless & Stateful Widgets**

- Stateful Widgets are used to build screens and Widgets (inside that will be refreshed and changed.

- The Widget is linked internally to state instance to build it on the screen.

$$\text{UI} = f(\text{state})$$

The layout
on the screen

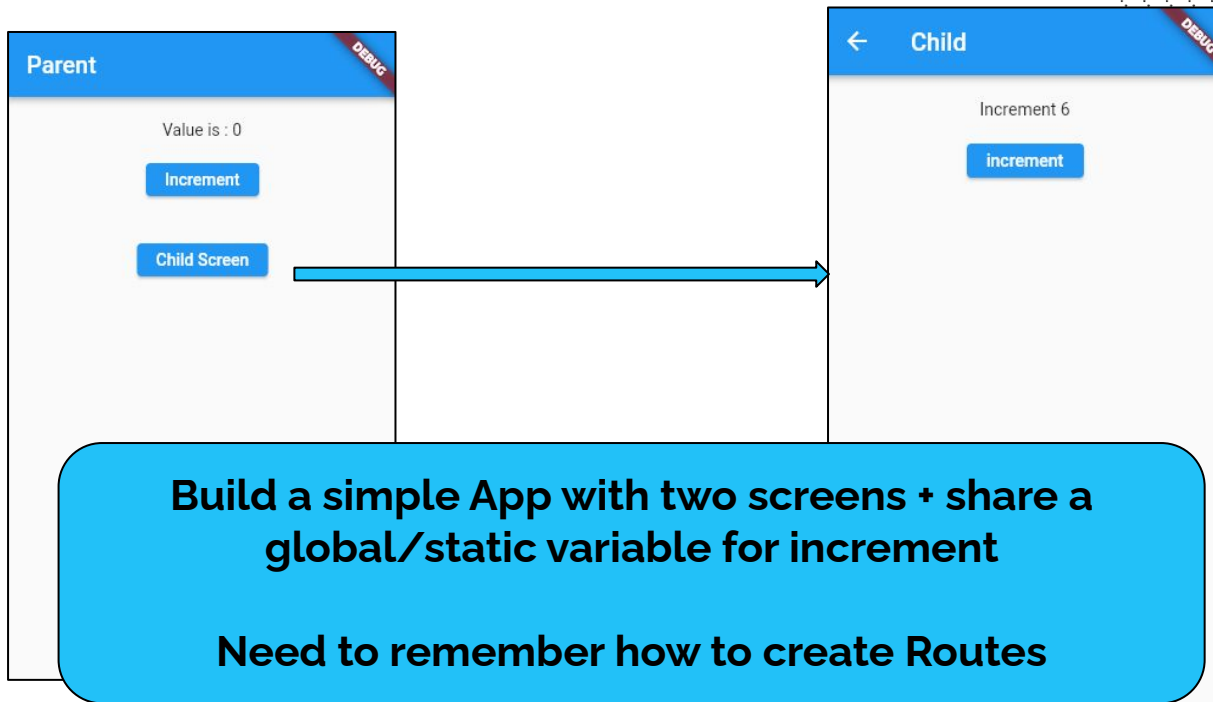
Your
build
methods

The application state

- There is no imperative changing of the UI itself (*like `widget.setText`*)—you change the state, and the UI rebuilds from scratch (Remember : of the relevant widget).
- Stateless Widgets are used to build static components that will not change.

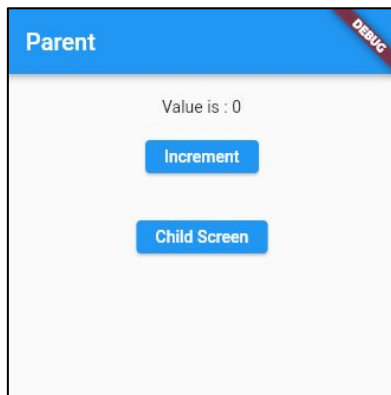
But , They can include stateful widgets which can change.

State Management in Flutter



State Management in Flutter

- Stateless & Stateful Widgets



```
import 'package:flutter/material.dart';
import
'package:state_management_hello/screens/child.dart';
import
'package:state_management_hello/screens/parent.dart';

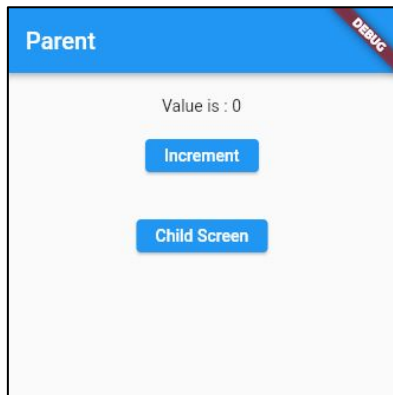
void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: ParentWidget(),
      routes: {
        "/home": (context) => ParentWidget(),
        "/child-screen": (context) => MyWidget(),
      },
    );
  }
}
```

State Management in Flutter

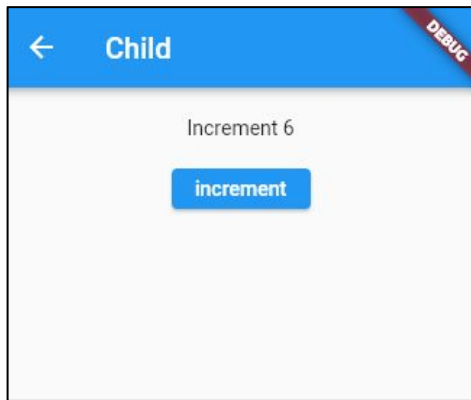
- Stateless & Stateful Widgets



```
class ParentWidget extends StatefulWidget {
  const ParentWidget ({super.key});
  static int increment = 1;
  @override
  State<ParentWidget> createState () => _MyWidgetState ();
}
class _MyWidgetState extends State<ParentWidget> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Parent")),
      body: Center(
        child: Column(
          children: [
            SizedBox(height: 20,),
            Text("Value is : ${ParentWidget.increment}"),
            SizedBox(height: 20,),
            ElevatedButton (
              onPressed: () {
                ParentWidget.increment = ParentWidget.increment + 1;
                setState(() {});
              },
              child: Text("Increment")),
            SizedBox(height: 40),
            ElevatedButton (
              onPressed: () {
                Navigator.pushNamed(context, "/child-screen");
              },
              child: Text("Child Screen"))
          ],
        ),
      ),
    );
  }
}
```

State Management in Flutter

- Stateless & Stateful Widgets



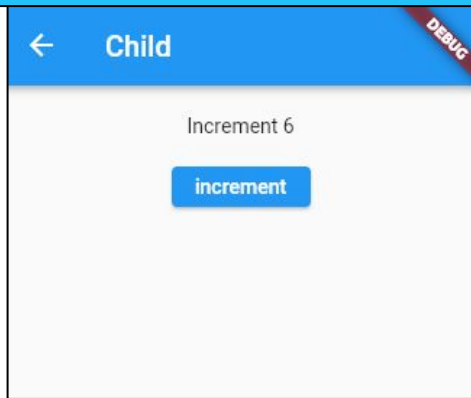
```
import 'package:flutter/material.dart';
import 'package:state_management_hello/screens/parent.dart';

class MyWidget extends StatefulWidget {
  const MyWidget({super.key});
  @override
  State<MyWidget> createState() => _MyWidgetState();
}

class _MyWidgetState extends State<MyWidget> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Child")),
      body: Center(
        child: Column(children: [
          SizedBox(
            height: 20,
          ),
          Text("Increment ${ParentWidget.increment}"),
          SizedBox(
            height: 20,
          ),
          ElevatedButton(
            onPressed: () {
              setState(() {
                ParentWidget.increment = ParentWidget.increment +
1;
              });
            },
            child: Text("increment")
          ]
        ]
      )
    );
  }
}
```

State Management Flutter

Parent class is coupled with this Class.



```
import 'package:flutter/material.dart';
import 'package:state_management_hello/screens/parent.dart';

class MyWidget extends StatefulWidget {
  const MyWidget({super.key});
  @override
  State<MyWidget> createState() => _MyWidgetState();
}

class _MyWidgetState extends State<MyWidget> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Child")),
      body: Center(
        child: Column(children: [
          SizedBox(
            height: 20,
          ),
          Text("Increment ${ParentWidget.increment}"),
          SizedBox(
            height: 20,
          ),
          ElevatedButton(
            onPressed: () {
              setState(() {
                ParentWidget.increment = ParentWidget.increment +
1;
              });
            },
            child: Text("increment")
          )
        ]
      )
    );
  }
}
```

State Management in Flutter

- Stateless & Stateful Widgets

Hit the Back Button :

On the parent Screen, the increment is refreshed or Not ?

```
import 'package:flutter/material.dart';
import 'package:state_management_hello/screens/parent.dart';
```

```
class MyWidget extends StatefulWidget {
  const MyWidget({super.key});

  @override
  State<MyWidget> createState() => _MyWidgetState();
}
```

```
class MyWidgetState extends State<MyWidget> {
  @override
  Widget build(BuildContext context) {
```

```
    Child()),
```

```
    ParentWidget.increment}"),
```

```
    increment = ParentWidget.increment + 1;
```

```
  ));
```

```
},
```

```
child: Text("increment")
```

```
])));
```

```
}
```

```
}
```

**No, it is not updated .
Build method is not called for the case of
“onResume”...**

```
import 'package:flutter/material.dart';  
import 'package:state_management_hello/screens/parent.dart';
```

```
StatefulWidget {  
  // ...  
});
```

```
State() => _MyWidgetState();
```

```
State<MyWidget> {
```

```
  context) {
```

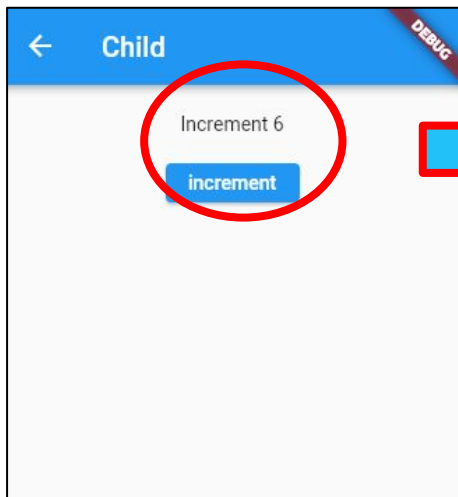
```
    child: Text("Child"),
```

```
    child: Column(children: [  
      SizedBox(  
        height: 20,
```

```
    ], mainAxisAlignment: MainAxisAlignment.spaceBetween),
```

```
    ParentWidget.increment = ParentWidget.increment + 1;
```

```
    ParentWidget.increment))
```



State Manage Flutter

- Stateless & Stateful Widgets

How to listen/refresh when the child activity completes or it is "killed"

then function : called on future objects. Its block will be executed when the future is returned/assigned

_ : a parameter not bothering to give it a name because it will not be used.

```
import 'package:flutter/material.dart';
class ParentWidget extends StatefulWidget {
  const ParentWidget({super.key});
  static int increment = 1;
  @override
  State<ParentWidget> createState() => _MyWidgetState();
}
class MyWidgetState extends State<ParentWidget> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Parent")),
      body: Center(
        child: Column(
          children: [
            SizedBox(height: 20,),
            Text("Value is : ${ParentWidget.increment}"),
            SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {
                ParentWidget.increment = ParentWidget.increment + 1;
                setState(() {});
              },
              child: Text("Increment")),
            SizedBox(height: 40),
            ElevatedButton(
              onPressed: () {
                Navigator.pushNamed(context, "/child-screen")
                  .then((_) => setState(() {}));
              },
              child: Text("Child Screen"))
          ],
        ),
      ),
    );
  }
}
```

State Management in

```
Navigator.pushNamed(context, "/child-screen")
```

Flutter



- St

```
Navigator.pushNamed(context, "/child-screen").then((_) => setState(() {}));
```

Or use async and await

```
onPressed: () async {  
  await Navigator.pushNamed(context, "/child-screen");  
  setState(() {});  
},
```



Widgets Impacting Each others

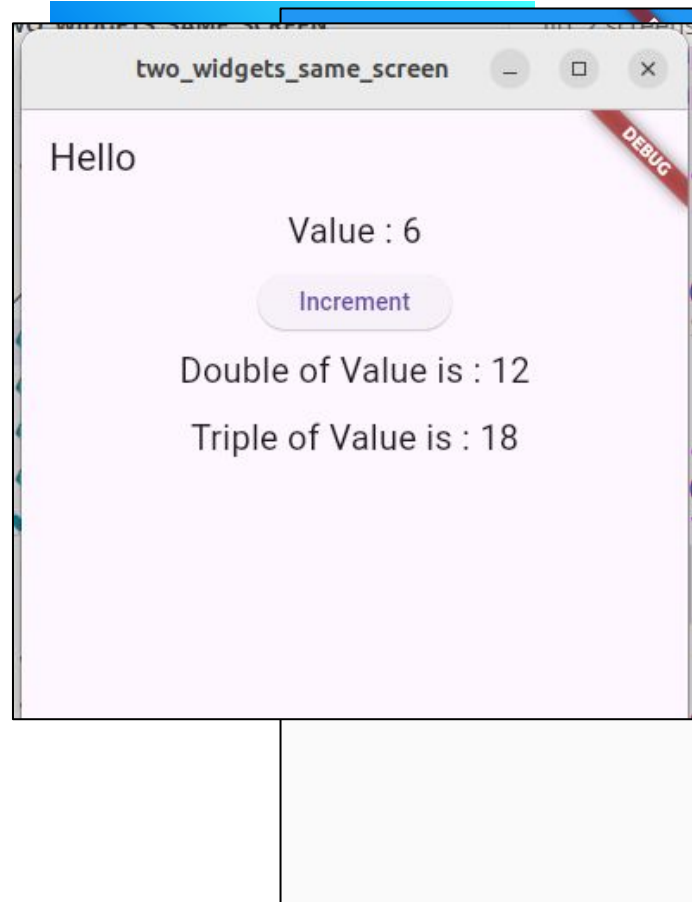
State Management in Flutter

- **Use Case Description**

- Single Screen:

- Three **SEPARATE** widgets:

- IncrementWidget
 - DoubleWidget
 - TrippleWidget

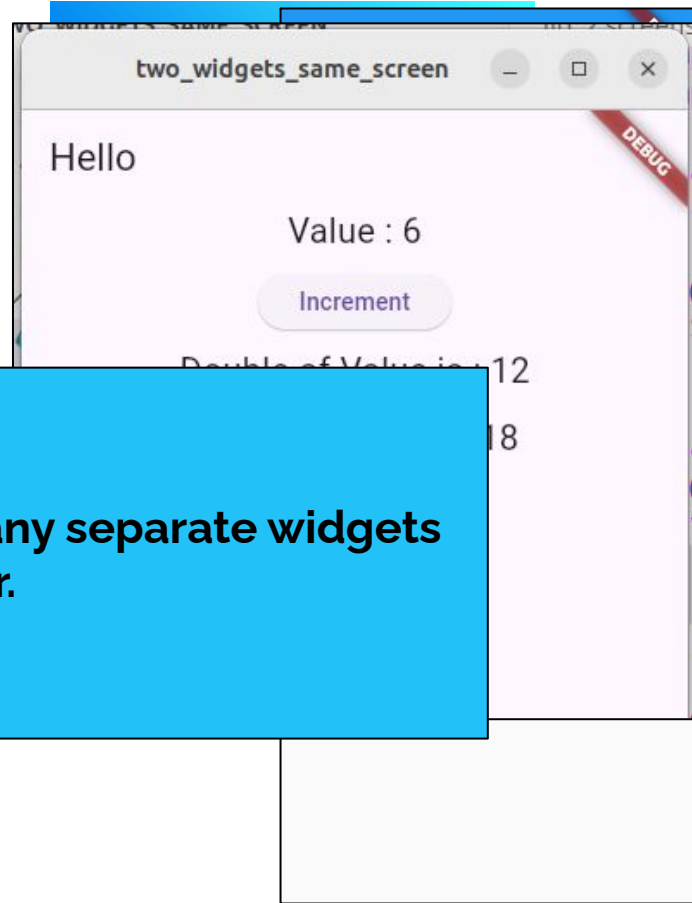


State Management in Flutter

- **Use Case Description**

- Single Screen:

For real apps, a screen contains always many separate widgets they impact each other.



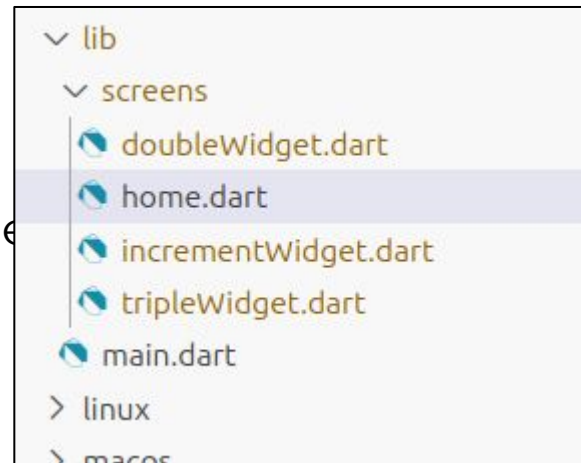
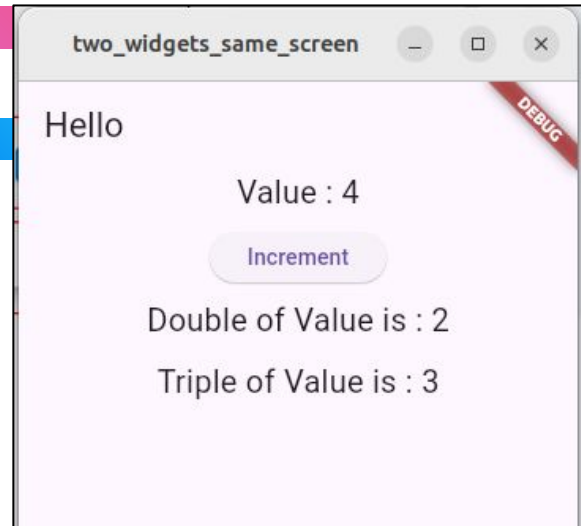
State Management in

```
import 'package:flutter/material.dart';
import 'package:two_widgets_same_screen/screens/home.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: HomeScreen(),
    );
  }
}
```



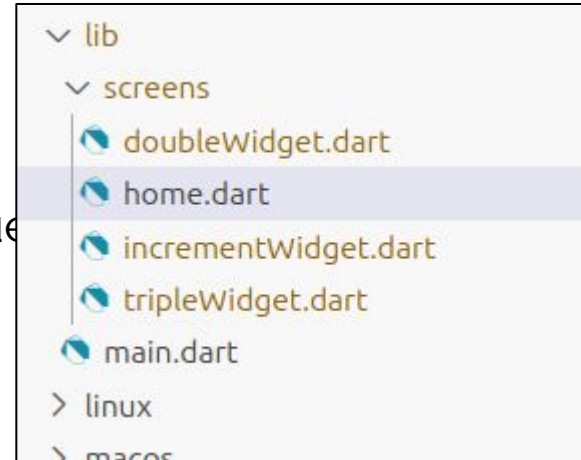
```

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class HomeScreenState extends State<HomeScreen> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Hello')),
      body: Center(
        child: Column(children: [
          IncrementWidget(),
          SizedBox(
            height: 10,
          ),
          DoubleWidget(),
          SizedBox(
            height: 10,
          ),
          TripleWidget()
        ]),
      );
  }
}

```



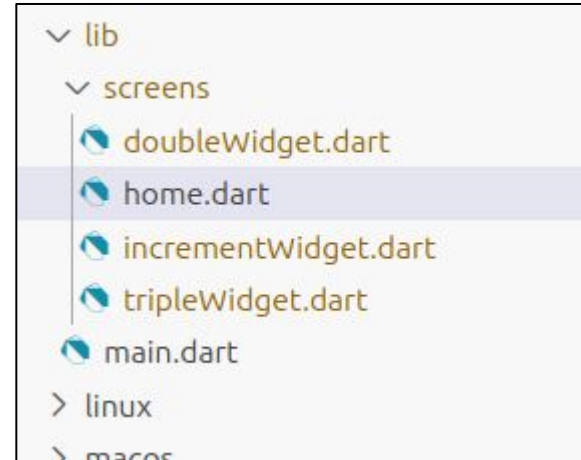
```

class IncrementWidget extends StatefulWidget {
  const IncrementWidget({super.key});
  static var increment = 1;
  @override
  State<IncrementWidget> createState() => _WidgetAState();
}

class WidgetAState extends State<IncrementWidget> {
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text("Value : ${IncrementWidget.increment}",
          style: TextStyle(fontSize: 20)),
        SizedBox(height: 10),
        ElevatedButton(onPressed: processIncrement,
          child: Text("Increment")),
      ],
    );
  }

  bool processIncrement() {
    IncrementWidget.increment = IncrementWidget.increment + 1;
    setState(() {});
    return true;
  }
}

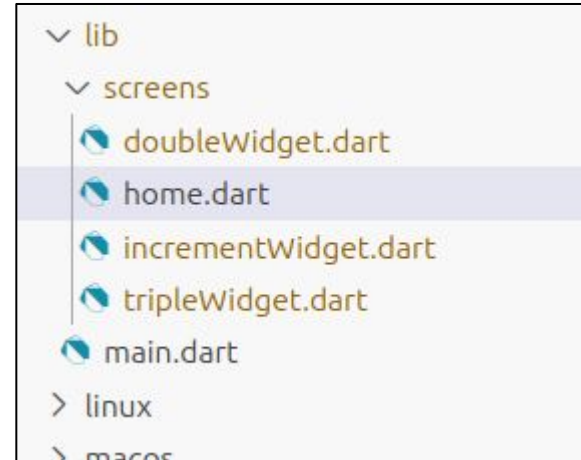
```



Will the second widget get updated too ?

```
class IncrementWidget {
  const IncrementWidget({Key? key}) : super(key: key);
  @override
  State<IncrementWidget> createState() {
    return _IncrementWidgetState();
  }
  class _IncrementWidgetState extends State<IncrementWidget> {
    int increment = 0;
    children: [
      Text("Value : ${IncrementWidget.increment}",
        style: TextStyle(fontSize: 20)),
      SizedBox(height: 10),
      ElevatedButton(onPressed: processIncrement,
        child: Text("Increment")),
    ],
  );
}

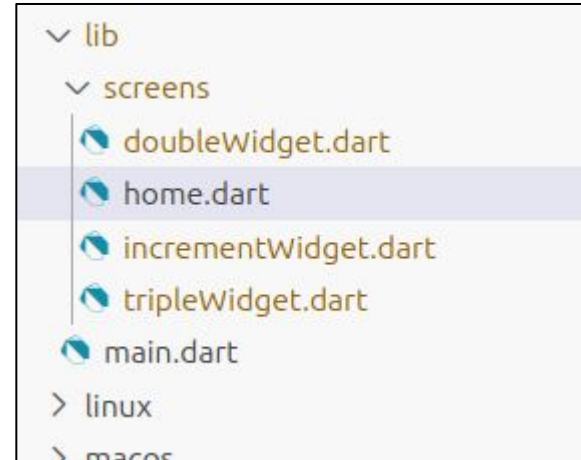
bool processIncrement() {
  IncrementWidget.increment = IncrementWidget.increment + 1;
  setState(() {});
  return true;
}
```



```

class DoubleWidget extends StatefulWidget {
  const DoubleWidget({super.key});
  @override
  State<DoubleWidget> createState() => _WidgetAState();
}
class _WidgetAState extends State<DoubleWidget> {
  @override
  void initState() {
    super.initState();
  }
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text(
          "Double of Value is : ${2 * IncrementWidget.increment}",
          style: TextStyle(
            fontSize: 20,
          ),
        ),
      ],
    );
  }
}

```



```

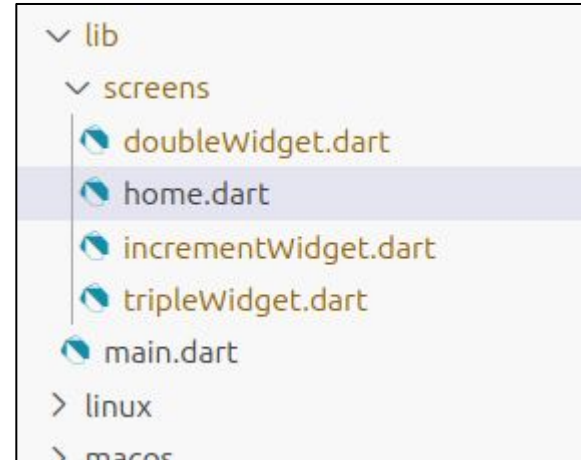
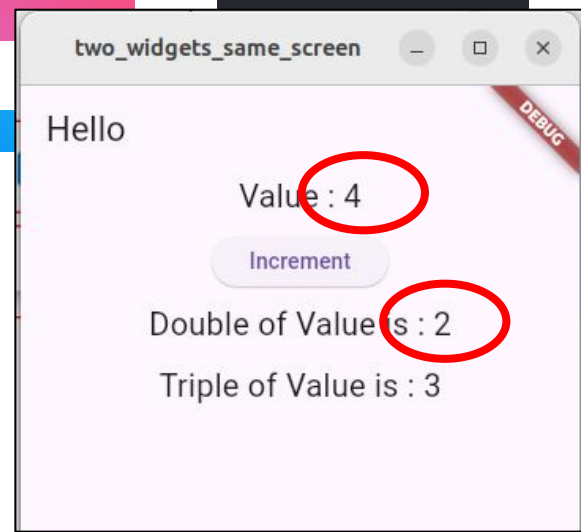
class DoubleWidget extends StatefulWidget {
  const DoubleWidget({super.key});
  @override
  State<DoubleWidget> createState() => _WidgetAState();
}
class _WidgetAState extends State<DoubleWidget> {
  @override
  void initState() {
    super.initState();
  }
  @override
  Widget build(BuildContext context) {
    return child;
  }
}

```

No, calling the setState(){} will only update the calling widget, but not other Widgets.

Any Solution ?

Don't think to put them into a single Widget.



State Management in Flutter



- **State Management**

- You have data (Variables or any data structure) that is usually used to draw the UI using Flutter.
 - State Management Techniques are used to ensure that data is fully synced with the UI.
 - If there is a change in the data, the UI is **automatically updated.** and vice versa.

State Management in Flutter

- **State Management Libraries**

- setState (Low-Level)
- Provider
- GetX
- Redux
- Bloc
- ...

Which better one to use ?

State Management in Flutter

- **State Management Libraries**

- setState (Low-Level)
- Provider
- GetX
- Redux
- BloC
- ...

**Can you solve this
problem yourself
without using any
library ?**

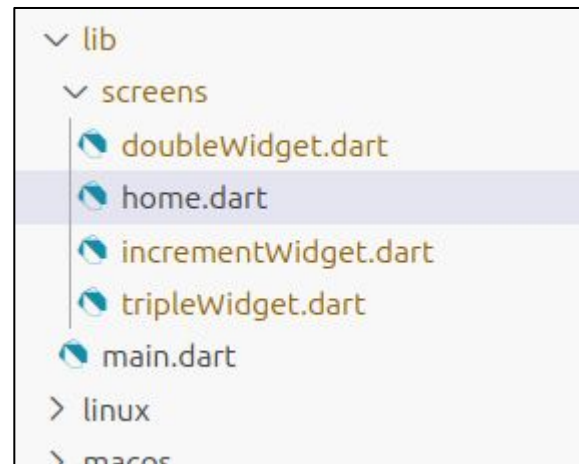
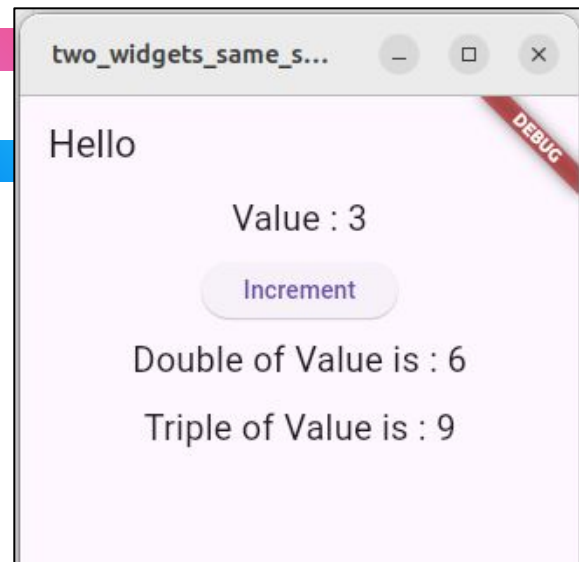
```

List<State> listners = [];
Future<bool> notify_listners() async {
  listners.forEach((item) {
    item.setState(() {});
  });
  return true;
}

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});
  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Hello')),
      body: Center(
        child: Column(children: [
          IncrementWidget(),
          SizedBox(height: 10),
          DoubleWidget(),
          SizedBox(height: 10),
          Triplewidget()
        ]),
      ));
  }
}

```



```

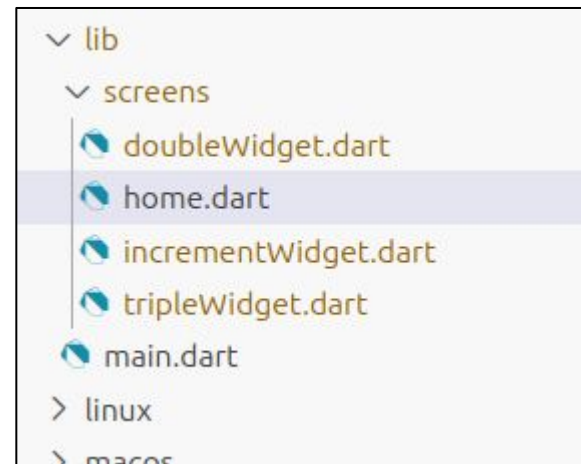
class DoubleWidget extends StatefulWidget {
  const DoubleWidget({super.key});

  @override
  State<DoubleWidget> createState() => _WidgetAState();
}

class _WidgetAState extends State<DoubleWidget> {
  @override
  void initState() {
    super.initState();
    listeners.add(this);
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text(
          "Double of Value is :  $\{2 * \text{IncrementWidget.increment}\}$ ",
          style: TextStyle(
            fontSize: 20,
          ),
        ),
      ],
    );
  }
}

```



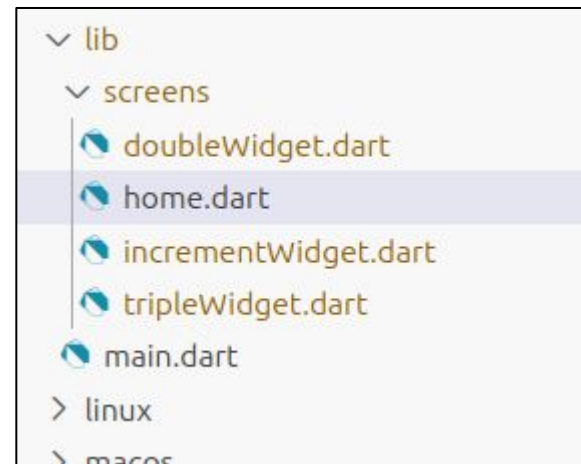
```

class IncrementWidget extends StatefulWidget {
  const IncrementWidget({super.key});
  static var increment = 1;
  @override
  State<IncrementWidget> createState() => _WidgetAState();
}

class WidgetAState extends State<IncrementWidget> {
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text("Value : ${IncrementWidget.increment}",
          style: TextStyle(fontSize: 20)),
        SizedBox(height: 10),
        ElevatedButton(onPressed: processIncrement,
          child: Text("Increment")),
      ],
    );
  }

  bool processIncrement() {
    IncrementWidget.increment = IncrementWidget.increment + 1;
    setState(() {});
    notify_listeners();
    return true;
  }
}

```



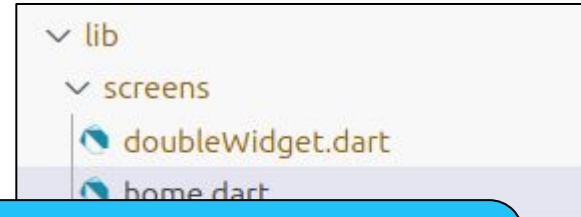
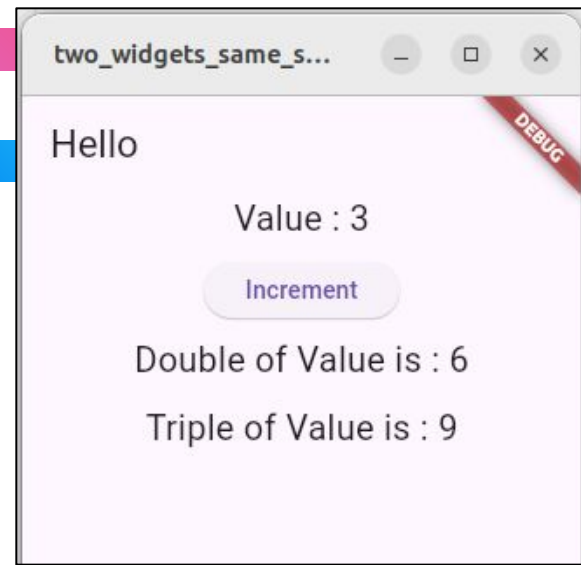
```

class IncrementWidget extends StatefulWidget {
  const IncrementWidget({super.key});
  static var increment = 1;
  @override
  State<IncrementWidget> createState() => _WidgetAState();
}

class _WidgetAState extends State<IncrementWidget> {
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text("Value : ${IncrementWidget.increment}",
          style: TextStyle(fontSize: 20)),
        SizedBox(height: 10),
        ElevatedButton(onPressed: processIncrement,
          child: Text("Increment")),
      ],
    );
  }

  bool processIncrement() {
    IncrementWidget.increment = IncrementWidget.increment + 1;
    setState(() {});
    notify_listeners();
    return true;
  }
}

```



Which design Pattern ?

Section 3

Other State Management Libraries in Flutter





GetX

State Management in Flutter



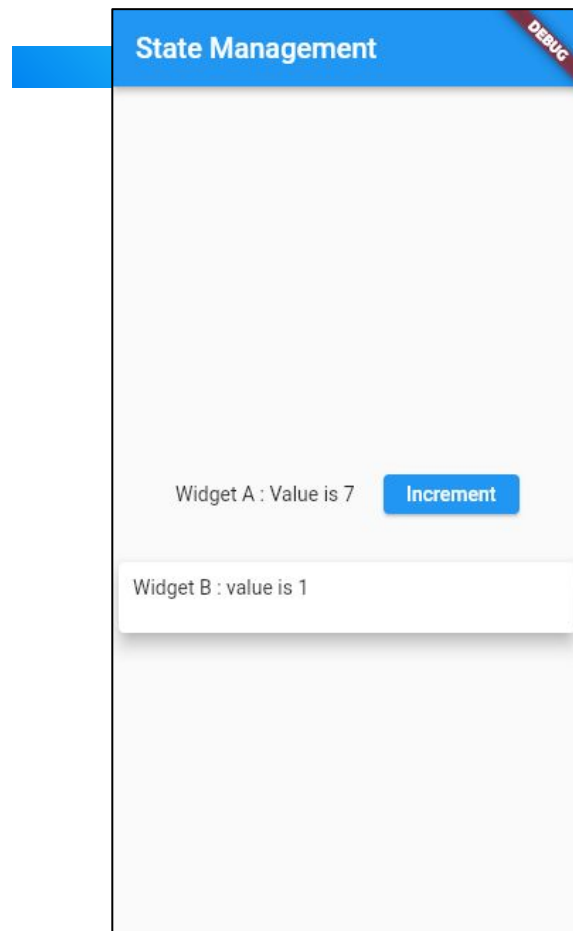
- **GetX**

- Claiming to be the most popular library within the flutter pub.dev
- Set of libraries offering various functionalities including:
 - State Management
 - Routing and Navigation
 - ...
- Why GetX : Performance + Coding Productivity + Decoupling

State Management in Flutter

- **GetX**

- How to use GetX state management to update multiple widgets within the same screen ?



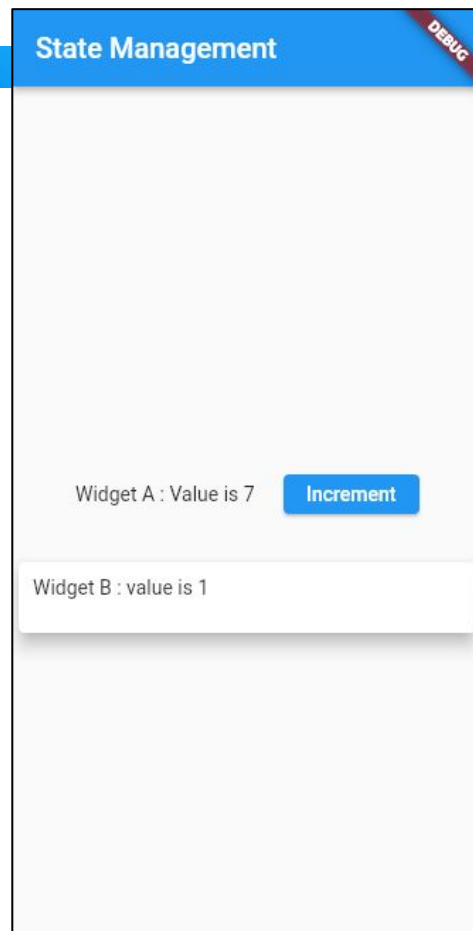
State Management in Flutter

- **GetX**

- From the command line, import the get Package

flutter pub add get

- Or, you can edit the pubspec.yaml file to add the relevant dependency



State Management in Flutter

```
import 'package:get/get.dart';

class IncrementController extends GetxController {
  final increment = 0.obs;

  @override
  void onInit() {
    super.onInit();
  }

  @override
  void onReady() {}

  @override
  void onClose() {}

  void doIncrement() {
    increment.value = increment.value + 1;
  }
}
```

State Management

DEBUG

Widget A : Value is 7

Increment

Widget B : value is 1

State Management in Flutter

```
import 'package:get/get.dart';

class IncrementController extends GetxController {
  final increment = 0.obs;

  @override
  void onInit() {
    super.onInit();
  }

  @override
  void onReady() {}

  @override
  void onClose() {}

  void doIncrement() {
    increment.value = increment.value + 1;
  }
}
```

Obs : To be observed for a change

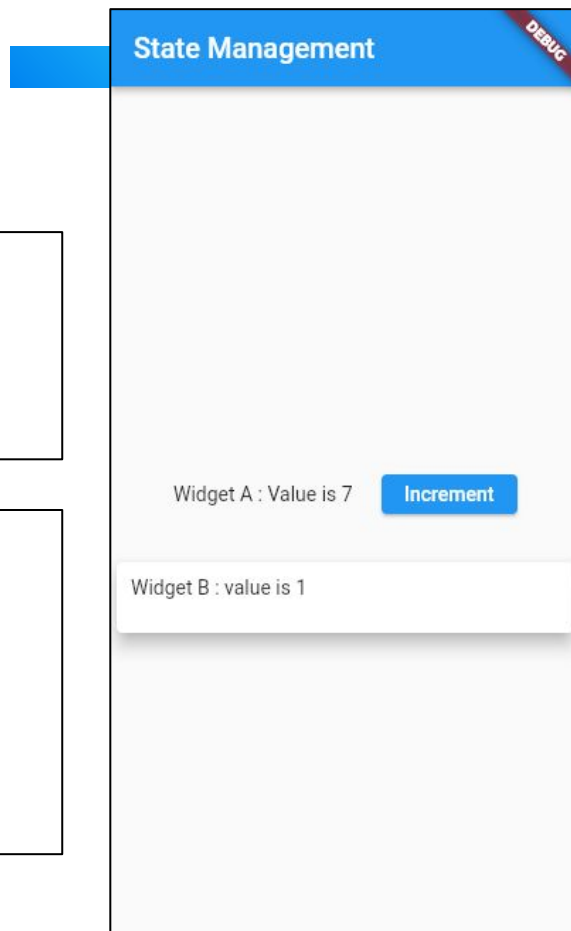
State Management

DEBUG

State Management in Flutter

```
final increment = 0.obs;  
final name = 'Hello'.obs;  
final myList = [].obs;  
final myMap = {'name': 'Amel'}.obs;
```

```
import 'package:get/get.dart';  
  
class MyListController extends GetxController {  
  final myList = [].obs;  
  
  void add(String name) {  
    myList.add(name);  
  }  
}
```



```

import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'controllers/increment.dart';
import 'widgets/myA.dart';
import 'widgets/myB.dart';
void main() {
  runApp(MainApp());
}
class MainApp extends StatelessWidget {
  MainApp({super.key});
  final IncrementController controller = Get.put(IncrementController());
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text("State Management")),
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              MyA(controller: controller),
              SizedBox(height: 30),
              MyB(controller: controller)
            ],
          ),
        ),
      ),
    );
  }
}

```

State Management

DEBUG

Widget A : Value is 7

Increment

Widget B : value is 1

```

import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/increment.dart';
import '../main.dart';

class MyA extends StatelessWidget {
  IncrementController controller;
  MyA({super.key, required this.controller});

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Obx(() =>
          Text("Widget A : Value is ${controller.increment}"),
        ),
        SizedBox(width: 20),
        ElevatedButton(
          onPressed: () {
            controller.doIncrement();
          },
          child: Text("Increment"))
      ],
    );
  }
}

```

State Management

DEBUG

Widget A : Value is 7

Increment

Widget B : value is 1

```

import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/increment.dart';
import '../main.dart';

class MyA extends StatelessWidget {
  IncrementController controller;
  MyA({super.key, required this.controller});

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Obx(() =>
          Text("Widget A : Value is ${controller.increment}")),
        SizedBox(width: 20),
        ElevatedButton(
          onPressed: () {
            controller.doIncrement();
          },
          child: Text("Increment"))
      ],
    );
  }
}

```

State Management

DEBUG

Widget A : Value is 7

Increment

Widget B : value is 1

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/increment.dart';
import '../main.dart';

class MyB extends StatelessWidget {
  IncrementController controller;
  MyB({super.key, required this.controller});

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Container(
        width: double.infinity,
        height: 30,
        margin: EdgeInsets.all(10),
        child:
          Obx(() =>
            Text("Widget B : value is ${controller.increment}"))
          ),
      elevation: 10);
  }
}
```

State Management

DEBUG

Widget A : Value is 7

Increment

Widget B : value is 1

State Management in Flutter

- **Provider** : Part of the Flutter Foundation Framework (Supported by Google)
 - The same way as getX:
 - Controller instead of extending GetxController,
 - It needs to extend : **ChangeNotifier**

```
import 'package:flutter/material.dart';

class IncrementModel extends ChangeNotifier {
  static var increment = 0;
  void doIncrement() {
    increment = increment + 1;
    notifyListeners();
  }
}
```

State Management in Flutter

- **Provider** : Part of the Flutter Foundation Framework (Supported by Google)
 - The same way as getX:
 - Instead of using Obx (()=> Widget)
 - Use **Consumer<ControllerName>**

```
body: Consumer<IncrementModel>(
  builder: (context, myModel, child) => Center(
    child: Column(
      children: [
        SizedBox(
          height: 20,
        ),
        Text("Value is : ${IncrementModel.increment}"),
```



Cubit / BloC

BloC

- **Definition**

- Bloc = **Business Logic Component**
 - is a powerful state management library in Flutter that helps separate business logic from the user interface.
- To use it inside **Flutter** :
 - `flutter pub add flutter_bloc`



BloC

- **Why to you must use BloC**

- **Clear Separation of Concerns:**

- Business logic is separate from UI, making code easier to maintain and test.

- **Reactivity:**

- Widgets automatically rebuild when a new state is **emitted**.

- **Powerful State Management:**

- States are managed in a structured, **predictable way** with possibility to share them across widgets, screens...

- **Testability:**

- Business logic is encapsulated in Bloc and can be tested independently of UI.

BloC

- **BloC Libraries**

- **Cubit**

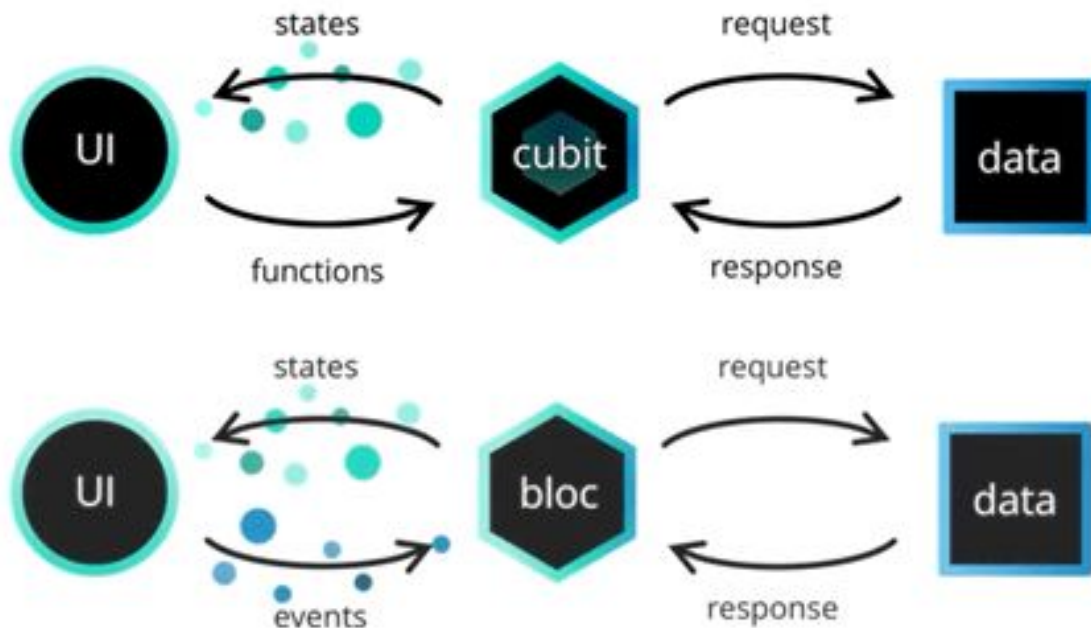
- A lightweight state management class for managing a single state.
 - Emits new states when triggered by methods.

- **Bloc:**

- Extends Cubit but introduces an event-based approach.
 - Reacts to incoming events and maps them to states.

BloC

- Flow of Actions



BLoC

- **Flow of Actions**

1. User Interaction: Begins with a user interaction
 - *User clicks "Increment" button*
2. Event Dispatch: This interaction dispatches an event to the BLoC.
 - *Within onPressed of the UI code, dispatch an event ..*
3. State Generation: The BLoC processes the event and yields a new state.
 - *State can be a new primitive value, an object ...*
4. UI Update: The UI listens to the BLoC and updates accordingly when a new state is emitted.

BloC

- BloC Widgets

- **BlocBuilder:**

- Like the FutureBuilder, it rebuilds a widget in response to state changes **continously**

- **BlocListener:**

- Reacts to state changes but does **NOT** rebuild widgets.

- **BlocConsumer:**

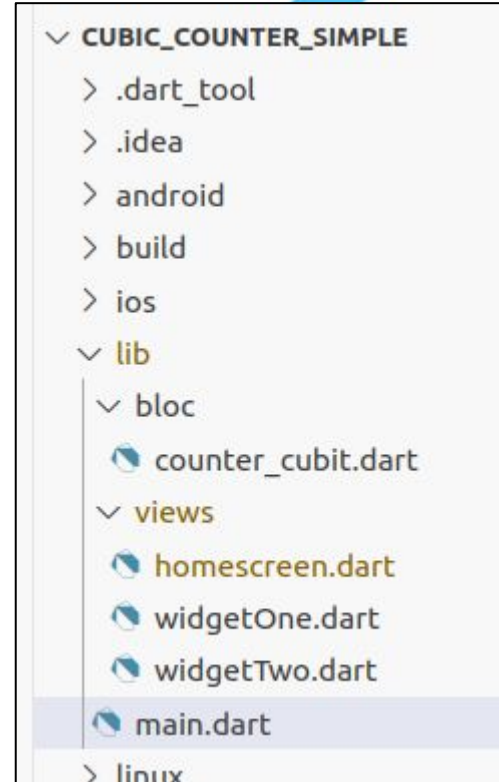
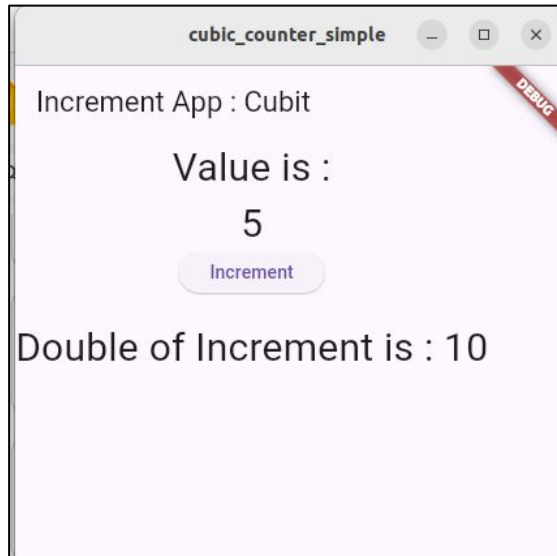
- **Combines** BlocBuilder and BlocListener for both rebuilding and reacting to states.

- **BlocProvider + MultiBlocProvider**

- Provides Bloc Objects to the widget tree

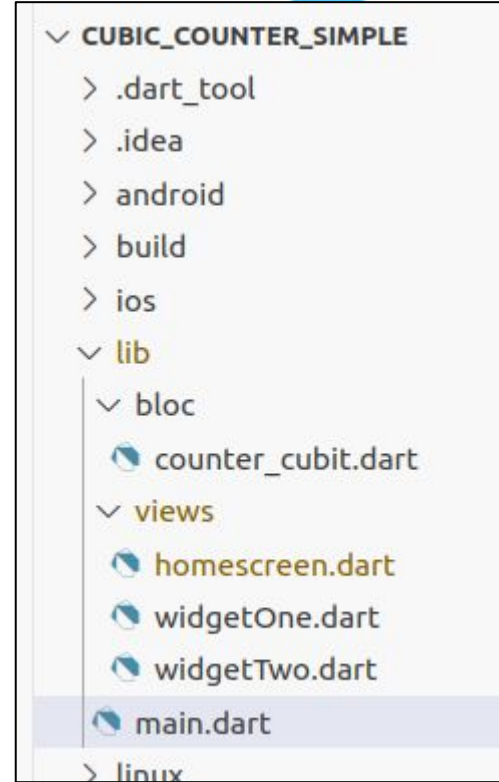
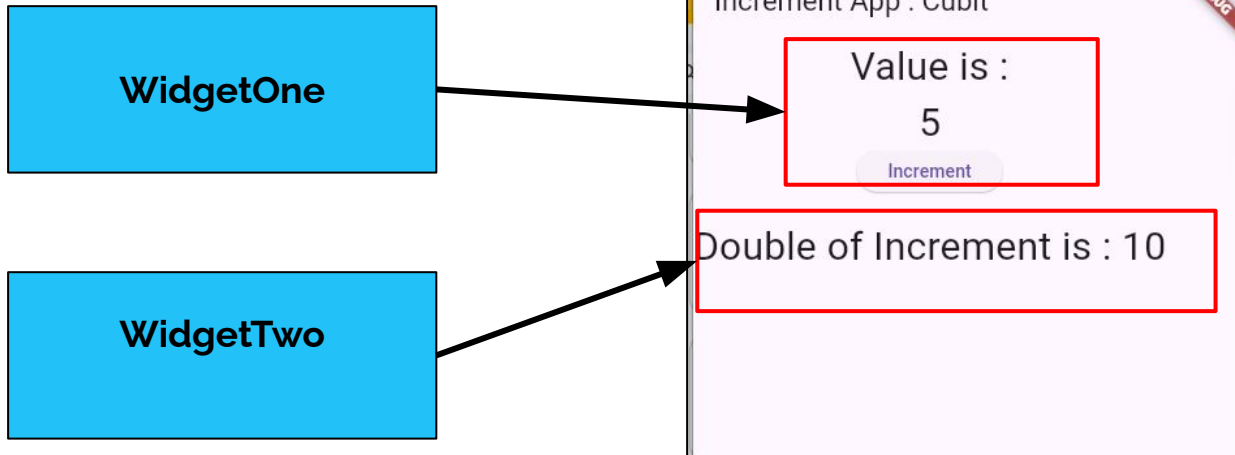
BloC

- Simple Incrementing App using BloC/Cubit



BloC

- Simple Incrementing App using BloC/Cubit

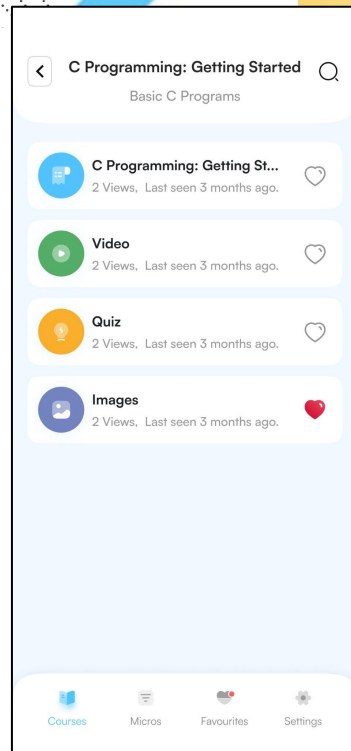
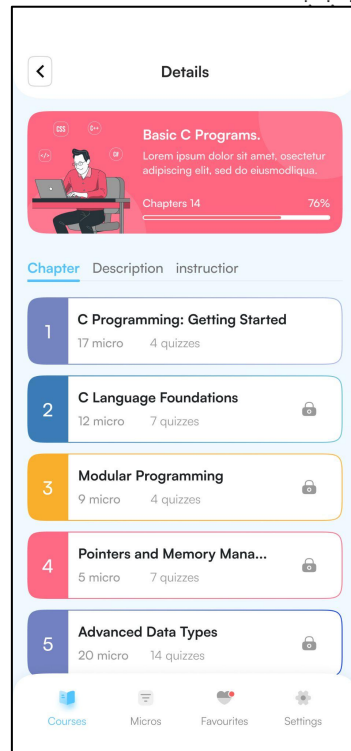
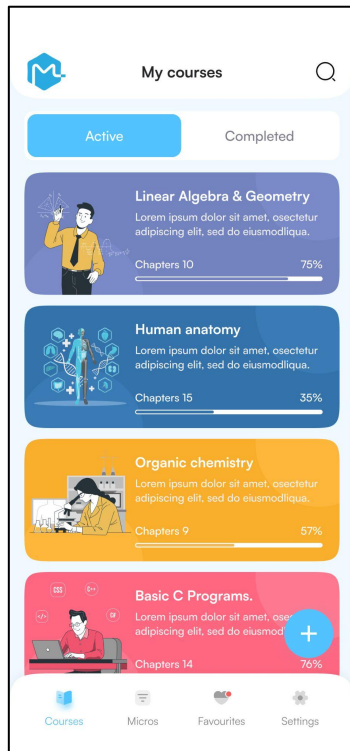
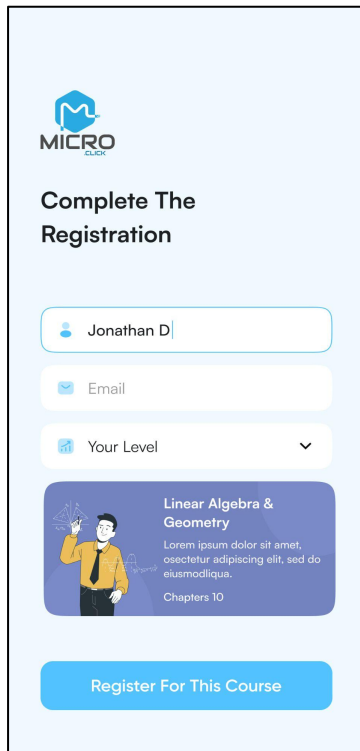
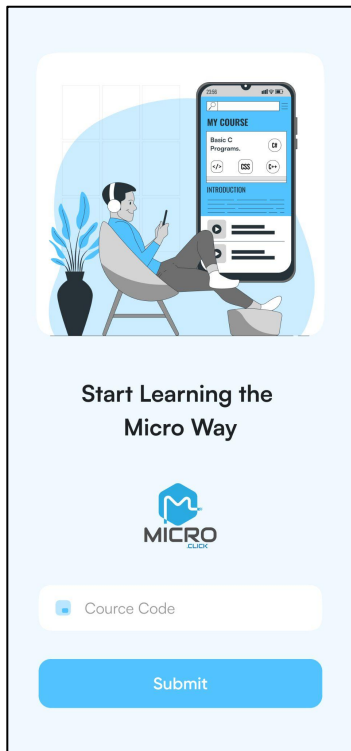


To delete

Building your MVP using Flutter



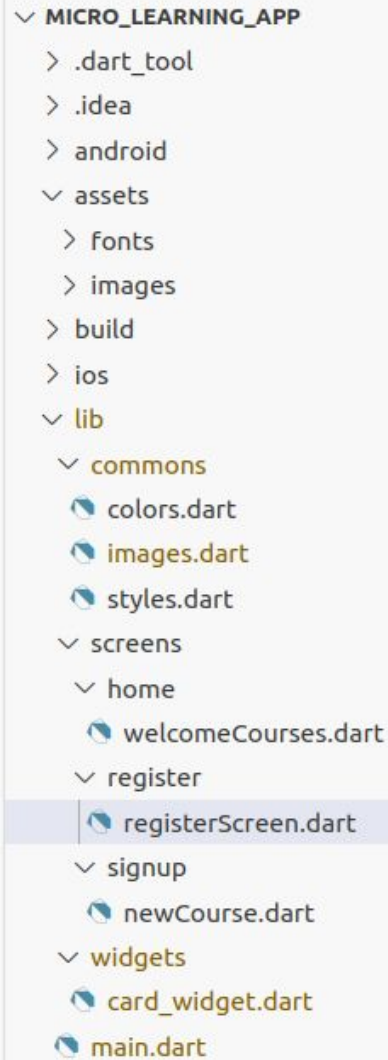
Building Your MVP using Flutter



Building Your MVP using Flutter

- Structuring the Project

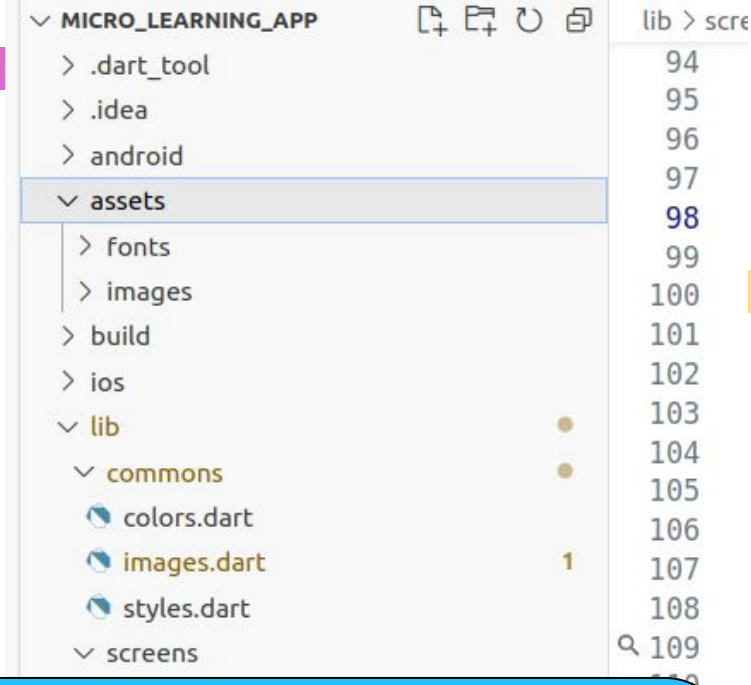
- assets
 - fonts
 - images
- lib
 - commons
 - colors.dart
 - screens
 - signup
 - signup.dart
 - bottomsection.dart
 - home
 - setting
 - widgets
 - navbar.dart
 - drawer.dart
- main.dart
- pubspec.yaml



100

- **Structuring the Project**

- **assets**
 - **fonts**
 - **images**
- **lib**
 - **commons**
 - colors.dart
 - **screens**
 - **signup**
 - signup.dart
 - bottomsection.d
 - **home**
 - **setting**
 - **widgets**
 - navbar.dart
 - drawer.dart
- main.dart
- pubspec.yaml



Better to have each screen inside its dedicated folder that can be used to store widgets related to the screen.

Widgets that are used/accessed by multiple screams, store them at lib/widgets

Building Your MVP using Flutter

- Structuring the Project

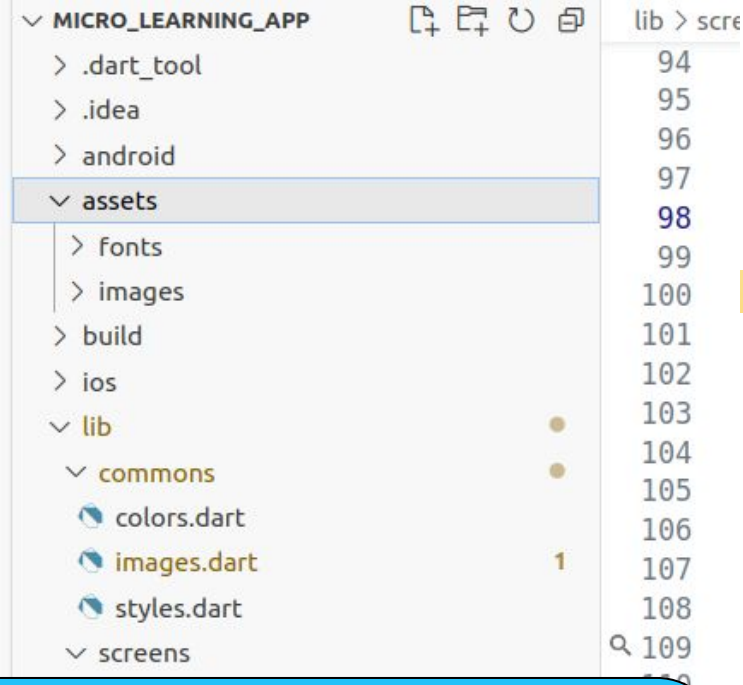
- lib / commons / styles.dart

```
import 'package:flutter/material.dart' ;
import 'colors.dart' ;

const startTextStyle = TextStyle (
  fontSize: 25.0,
  color: textColor,
  fontFamily: "Satoshi",
  fontWeight: FontWeight.w700,
  fontStyle: FontStyle.normal,
);
const buttonTextStyle = TextStyle (
  fontSize: 18.0,
  color: whiteColor,
  fontFamily: "Satoshi",
  fontWeight: FontWeight.w600,
  fontStyle: FontStyle.normal,
);
...
```

The benefit of having all styles/colors/Image URLs is to be better organized and have centralized point for changing the app.

(Same way as CSS classes inside a styles)



Building Your MVP using Flutter

- Structuring the Project

- lib / commons / colors.dart

```
import 'package:flutter/material.dart';

const backgroundColor = Color(0xffeff9ff);
const blackColor = Color(0xff000000);
const whiteColor = Color(0xffffffff);
const white1Color = Color(0xffff2f2f2);
const textColor = Color(0xff1e2123);
const text1Color = Color(0xff817f8c);
const brownColor = Color(0xff999999);
const unselectColor = Color(0xff868686);
const buttonColor = Color(0xff52c3ff);
const barTitleColor = Color(0xff252627);

const card1Color = Color(0xff7484c1);
const card2Color = Color(0xff3b7cb6);
const card3Color = Color(0xffff9af2c);
```

▼ MICRO_LEARNING_APP

> .dart_tool

> .idea

> android

▼ assets

> fonts

> images

> build

> ios

▼ lib

▼ commons

🔍 colors.dart

🔍 images.dart

🔍 styles.dart

▼ screens

▼ home

🔍 welcomeCourses.dart

▼ register

🔍 registerScreen.dart

▼ signup

🔍 newCourse.dart

▼ widgets

🔍 card_widget.dart

🔍 main.dart

Building Your MVP using Flutter

- Structuring the Project

- lib / commons / images.dart

```
import 'package:flutter/material.dart';

//////////////////////////////////////// Png Images

const splash_image = "assets/images/splash.png";
const logo_image = "assets/images/logo.png";
const elip_image = "assets/images/elip.png";
const men_image = "assets/images/men.png";
const anot_image = "assets/images/anot.png";
const cancer_image = "assets/images/cancer.png";
const code_image = "assets/images/code.png";
const logo_one_image = "assets/images/logo_one.png";
```

▼ MICRO_LEARNING_APP

> .dart_tool

> .idea

> android

▼ assets

> fonts

> images

> build

> ios

▼ lib

▼ commons

● colors.dart

● images.dart 1

● styles.dart

▼ screens

▼ home

● welcomeCourses.dart

▼ register

● registerScreen.dart

▼ signup

● newCourse.dart

▼ widgets

● card_widget.dart 1

● main.dart 2

Building Your MVP using Flutter

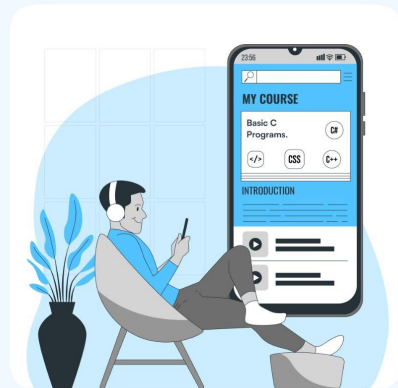
- Tree of Widgets :

- MaterialApp

- ?

- ?

- ?



Start Learning the
Micro Way

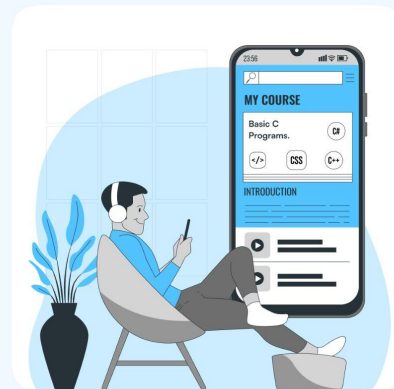


☐ Course Code

Submit

Building Your MVP using Flutter

- **Tree of Widgets :**
 - MaterialApp
 - Scaffold
 - Column
 - Image
 - Text
 - Image
 - TextFormField
 - ElevatedButton

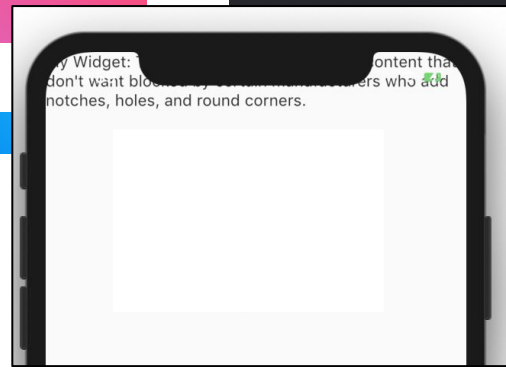


**Start Learning the
Micro Way**



Submit

Building Your MVP using Flutter

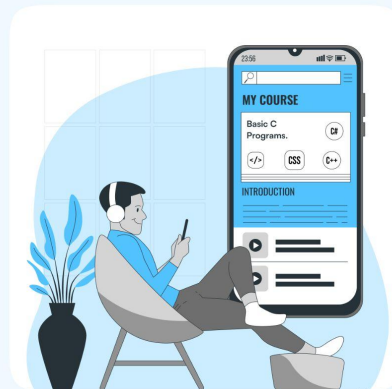


- **Tree of Widgets**

- **SafeArea** : A Container Widget that adds padding automatically to make sure that your widget is visible and hidden by the physical constraints.
- **InkWell** : Like GestureDetector but less options. Used for detecting touch events (OnTap ...)
- **Ink** : A Convenience Widget like a Container mostly used for decorating
- **SingleChildScrollView**
- **InputDecoration** : Used to decorate and style form widgets

Building Your MVP using Flutter

- **Tree of Widgets :**
 - MaterialApp
 - Scaffold
 - SafeArea
 - SingleChildScrollView
 - Column
 - Image
 - Text
 - Image
 - TextFormField
 - InputDecoration
 - ElevatedButton



**Start Learning the
Micro Way**

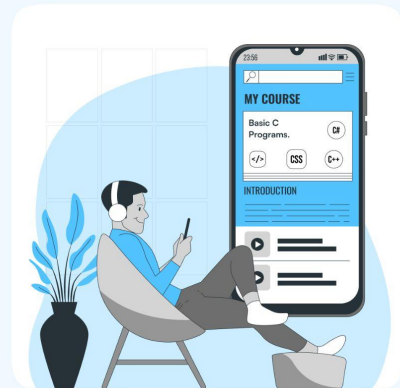


☐ Source Code

Submit

Building Your MVP using Flutter

- **Tree of Widgets :**
 - MaterialApp
 - Scaffold
 - SafeArea
 - SingleChildScrollView
 - Column
 - Image
 - Text
 - Image
 - TextFormField
 - InputDecoration
 - **InkWell**
 - **Ink**
 - **DecorationBox**
 - **Text**



Start Learning the
Micro Way



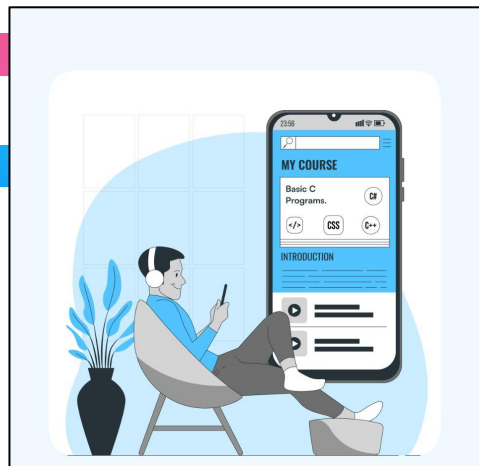
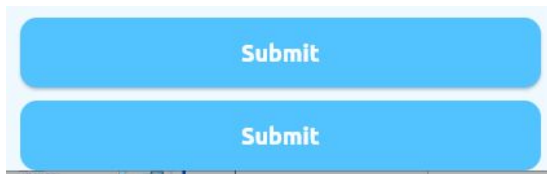
Source Code

Submit

```

Padding(
  padding: const EdgeInsets.symmetric(horizontal: 32.0),
  child: SizedBox(
    width: double.infinity,
    height: 54.0,
    child: ElevatedButton(
      onPressed: () {},
      style: ElevatedButton.styleFrom(
        backgroundColor: buttonColor,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(13),
        ),
        child: Text("Submit", style: buttonTextStyle),
      ),
    ),
  ),
),

```



```

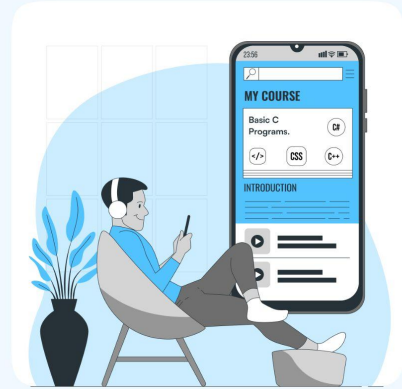
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 32.0),
  child: InkWell(
    onTap: () {},
    child: Ink(
      height: 54.0,
      width: size.width,
      decoration: BoxDecoration(
        color: buttonColor,
        borderRadius: BorderRadius.circular(13.0)),
      child: const Center(
        child: Text("Submit", style: buttonTextStyle),
      ),
    ),
  ),
),

```

```

Padding(
  padding: const EdgeInsets.symmetric(horizontal: 32.0),
  child: Container(
    height: 50.0,
    width: size.width,
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(13.0),
      color: whiteColor,
    ),
    child: TextFormField(
      cursorColor: brownColor,
      style: fieldTextStyle,
      decoration: const InputDecoration(
        border: InputBorder.none,
        focusedBorder: OutlineInputBorder(
          borderRadius: BorderRadius.all(Radius.circular(13.0)),
          borderSide: BorderSide(color: Colors.blue)),
        prefixIcon: SizedBox(
          height: 16.0,
          width: 16.0,
          child: Icon(Icons.key),
        ),
        hintText: "Course Code",
        hintStyle: codeTextStyle,
      ),
    ),
  ),
),
),
),
),

```



**Start Learning the
Micro Way**



 Course Code

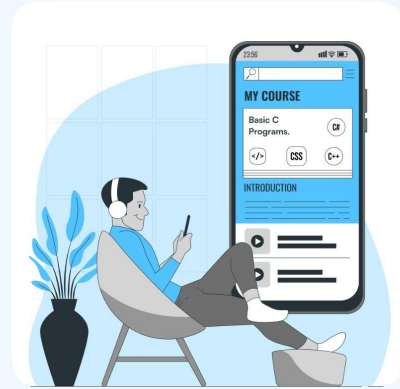
 Course Code

Submit

```

@override
Widget build(BuildContext context) {
  var size = MediaQuery.of(context).size;
  return Scaffold(
    backgroundColor: backColor,
    body: SafeArea(
      child: SingleChildScrollView(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.center,
          mainAxisAlignment: MainAxisAlignment.start,
          children: [
            SizedBox(
              height: size.height * 0.05,
            ),
            Padding(
              padding: const EdgeInsets.symmetric(horizontal: 32.0),
              child: Image.asset(
                splash_image,
                fit: BoxFit.fill,
              ),
            ),
            SizedBox(
              height: size.height * 0.05,
            ),
          ],
        ),
      ),
    ),
  );
}

```



**Start Learning the
Micro Way**



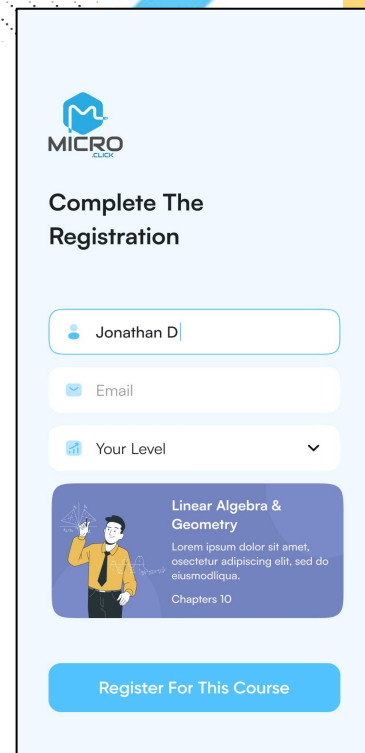
☐ Source Code

Submit

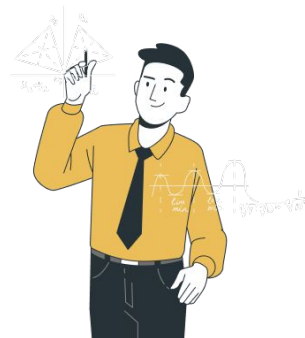
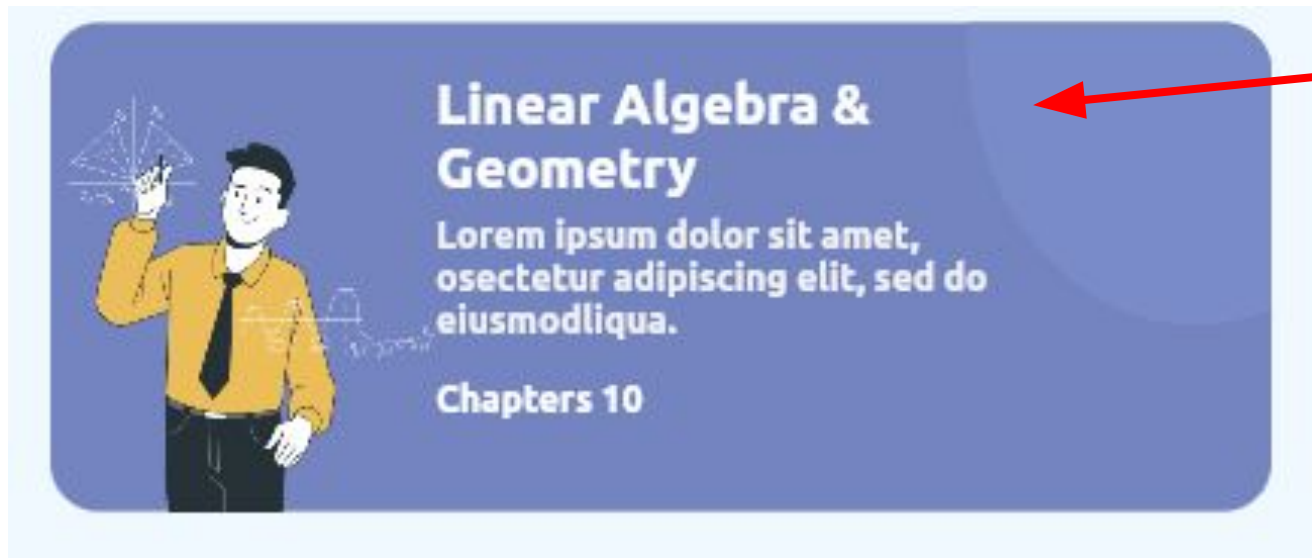
Building Your MVP using Flutter

- **Tree of Widgets :**

- MaterialApp
 - Scaffold
 - SafeArea
 - SingleChildScrollView
 - Column
 - SizedBox
 - Image
 - Text
 - TextFormField
 - InputDecoration
 - DropdownButtonFormField<String>
 - InputDecoration
 - ?
 - ?
 - ?

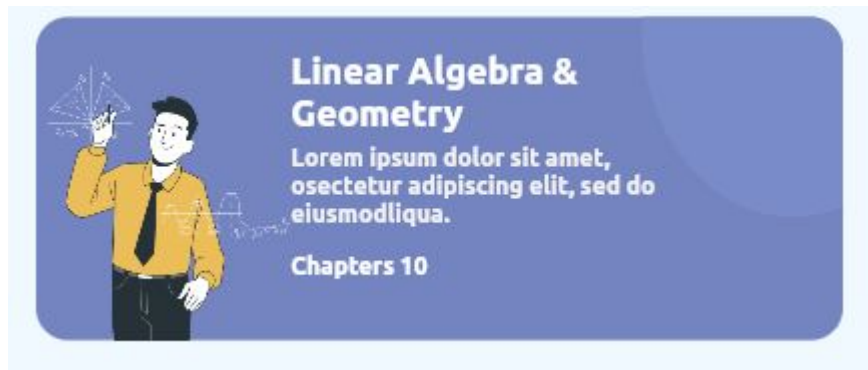


Building Your MVP using Flutter



Building Your MVP using Flutter

- **Tree of Widgets :**
 - Stack
 - Container
 - BoxDecoration
 - ClipRRect
 - Row
 - Image.asset
 - Container
 - BoxDecoration
 - Row
 - Image.asset
 - Column
 - Text
 - Text
 - Text



```

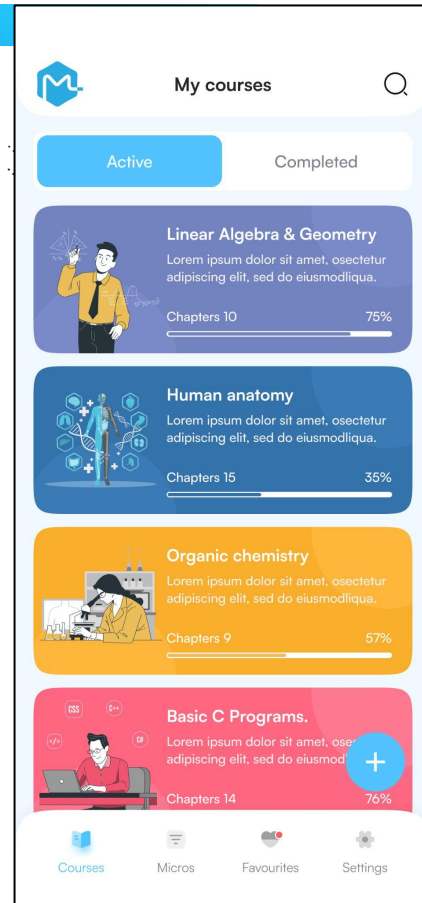
Stack(
  children: [
    Container(
      height: size.height * 0.2,
      width: size.width,
      decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(16.0),
        color: card1Color,
      ),
      child: ClipRRect(
        borderRadius: const BorderRadius.only(
          topRight: Radius.circular(16.0)),
        child: Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          mainAxisAlignment: MainAxisAlignment.end,
          children: [
            Image.asset(elip_image, height: 100),
          ],
        ),
      ),
    ),
    Container(
      height: size.height * 0.2,
      width: size.width,
      decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(16.0)),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.end,
        mainAxisAlignment: MainAxisAlignment.start,
        children: [
          Image.asset(men_image, height: size.height * 0.17),
          SizedBox(
            width: size.width / 2.15,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              mainAxisAlignment: MainAxisAlignment.start,
              children: [
                SizedBox(height: size.height * 0.02),
                const Text("Linear Algebra & Geometry", style: titleTextStyle),
                SizedBox(height: size.height * 0.007),
                const Text("Lorem ipsum dolor sit amet, osectetur adipiscing elit, sed do eiusmod aliqua. ",
                  style: descTextStyle),
                SizedBox(height: size.height * 0.015),
                const Text("Chapters 10", style: chapterTextStyle),
              ],
            ),
          ),
        ],
      ),
    ),
  ],
)

```

Building Your MVP using Flutter

- **Tree of Widgets :**
 - MaterialApp
 - Scaffold
 - ?

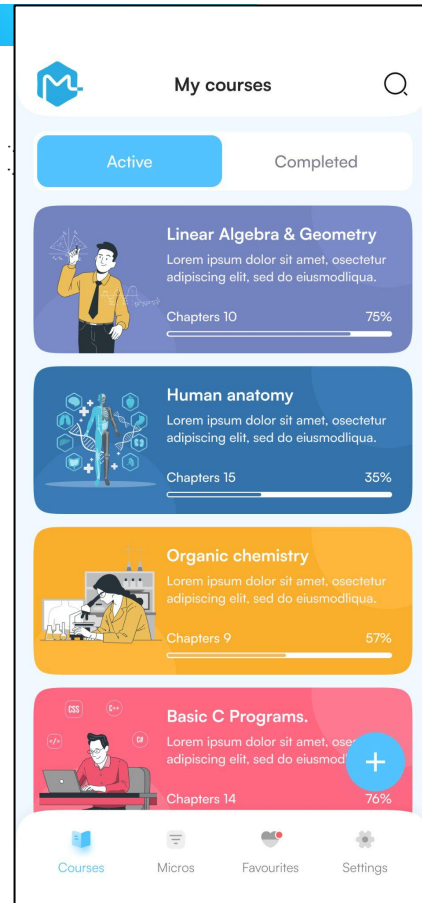
The user can slide from left to right between different pages



Building Your MVP using Flutter

- **Tree of Widgets :**

- MaterialApp
 - Scaffold
 - GetTopBarWidget ()
 - PageView
 - AllCourses()
 - TabBar
 - TabBarView
 - Column
 - Card
 - Card
 - MicroInformation()
 - Favorite()
 - Settings()
 - GetBottomNavBar()
 - FloatingActionButton



```

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: backColor,
    floatingActionButton: FloatingActionButton(
      onPressed: () {},
      backgroundColor: buttonColor,
      child: const Center(child: Icon(Icons.add, color: whiteColor, size: 30.0)),
    ),
    bottomNavigationBar: getBottomBarWidget(context),
    body: PageView(
      controller: pageController,
      onPageChanged: (index) {pageChangedViaSliding(index);},
      children: <Widget>[
        MyCourses(),
        MyMicros(),
        FavoriteWidget(),
        SettingWidget()],
    ),
  );
}

int bottomSelectedIndex = 0;
PageController pageController = PageController(
  initialPage: 0,
  keepPage: true,
);

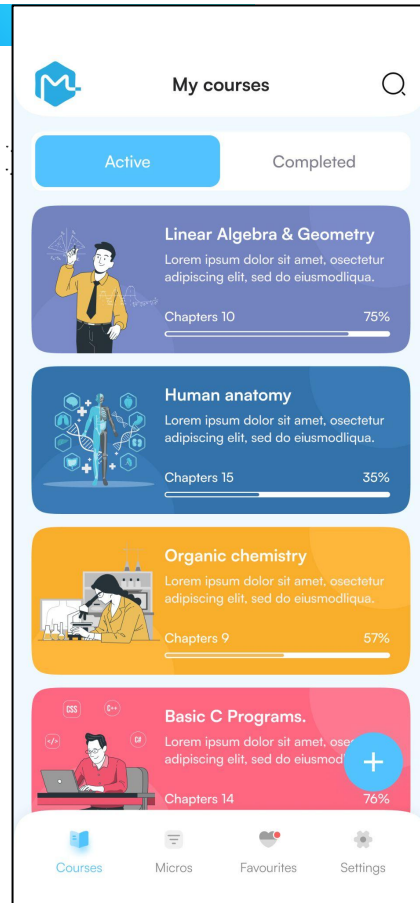
void bottomBarIsClicked(int index) {
  setState(() {
    bottomSelectedIndex = index;
    pageController.animateToPage(index,
      duration: Duration(milliseconds: 500), curve: Curves.ease);
  });
}

void pageChangedViaSliding(int index) {
  setState(() {bottomSelectedIndex = index;});
}

```

Building Your MVP using Flutter

- Widgets to Build using Flutter



Building Your MVP using Flutter

